UNIVERSITATEA BABEŞ-BOLYAI

Facultatea de Ştiinţe Economice şi Gestiunea Afacerilor

Informatică Economică

Lucrare de licenţă

Aplicație web pentru digitalizarea proceselor administrative ale unui cabinet de psihologie

Coordonator ştiinţific,

Lector univ. dr. Florina **COVACI**

2025

Rezumat

Această lucrare prezintă etapele dezvoltării unei aplicații web destinate digitalizării aspectelor administrative necesare funcționării eficiente a unui cabinet de psihologie. Pornind de la analiza pieței concurențiale și a contextului informatizării accelerate, am constatat că prezența în mediul *online* devine o cerință obligatorie pentru toate afacerile care doresc să supraviețuiască în acest mediu din ce în ce mai competitiv. Practic, obiectivul principal al acestui demers a fost reducerea volumului muncii adminstrative pe care beneficiarul (psihologul) o are de făcut, consumând astfel o parte semnificativă a resurselor mentale pe sarcini care nu țin de munca efectivă a acestuia. Pentru materializarea acestei idei am folosit tehnologii/limbaje precum HTML, CSS, JavaScript, PHP și MySQL. Combinarea acestora mi-a permis să creez o interfață web aspectuoasă care să integreze, printre altele, și câteva funcționalități avansate precum programări/teste psihologile online, oferind o experiență de utilizare intuitivă atât pentru clienți cât și pentru administratori.

Cuprins

Lista abrevierilor **1**
Lista tabelelor și a figurilor **2**
Introducere **3**
1. Metodologia **6**
1.1. Aspecte generale de metodologie 6
1.2. Tehnologii specifice 10
2. Analiză și proiectare **13**
2.1. Analiza contextului de business 13
2.2. Motivația lucrării: 14
2.3. Cerințele și beneficiile implementării: 15
2.4. Elemente de proiectare a aplicației web 17
2.4.1. Considerații generale privind proiectarea software 17
2.4.2. Modelare conform standardelor UML si BPMN 17
2.5. Fluxurile de utilizare și logica aplicației 21
2.6. Sinteză și concluzii 23
3. Implementare **23**
3.1. Aspecte generale ale implementării 23
3.1.1. Transmiterea automata a unui email după fiecare programare online 24
3.1.2. Formatarea unor date pentru a fi mai ușor de citit de către utilizatori. 25
3.1.3. Generarea unor chart-uri pentru ilustrarea unor statistici 25
3.1.4. Inițierea unei sesiuni în momentul în care ne logăm ca administrator 26
3.1.5. Imposibilitatea ștergerii contului de administrator cu care ești logat 26
3.1.6. Extragerea datelor privind programarile pentur a genera o situație PDF. 27
3.1.7. Obținerea perioadelor disponibile pentru programare 27
3.1.8. Extragerea datelor privind rezultatele la teste pentru afișarea pe chart-uri. 28
3.1.9. Logica de autentificare în platforma de administrator *28*
3.1.10. Popularea perioadelor/datelor/orelor disponibile pentru programări 29
3.1.11. Limitarea intervalului disponibil pentru programări la 21 zile în viitor 30
3.1.12. Transmiterea datelor din formularul de programare către server prin Ajax 31
3.1.13. Generarea și procesarea testelor online 31
4. Evaluarea, testarea și validarea aplicației **34**
4.1. Aspecte și precizări generale 34
4.1.1. Importanța testării în procesul de dezvoltare 34
4.1.2. Testarea funcțională 35
4.1.3. Testarea logicii aplicației (back-end) 35
4.1.4. Testarea interfeței (front-end) 36
4.1.5. Erori frecvent întâlnite în timpul dezvoltării 37
4.2. Sinteză 38
Concluzii și direcții de viitor **39**
Bibliografie **43**
Anexe **44**

Lista abrevierilor

AJAX	-	*Asynchronous JavaScript and XML*
API	-	*Application Programming Interface*
BPMN	-	*Business Process Model and Notation*
CRUD	-	*Create-Read-Update-Delete*
CSS	-	*Cascading Style Sheets*
GUI	-	*Graphical User Interface*
DOM	-	*Document Object Model*
HTML	-	*HyperText Markup Language*
MVC	-	*Model-View-Controller*
OOP	-	*Object Oriented Programming*
PDO	-	*PHP Data Object*
PHP	-	*Hypertext Preprocessor*
SMTP	-	*Simple Mail Transfer Protocol*
SQL	-	*Structured Query Language*
UI/UX	-	*User Interface / User Experience*
UML	-	*Unified Modeling Language*
XML	-	*eXtensible Markup Language*

Lista tabelelor și a figurilor

Figura 1 - Diagrama Use-case *(pag. 5)*

Figura 2 - Diagrama de activități UML: before *(pag. 7)*

Figura 3 - Diagrama de activități UML*:* after *(pag. 8)*

Figura 4 - Tabel de analiză comparativă (before&after) *(pag. 9)*

Figura 5 - Diagrama de deployment *(pag. 12)*

Figura 6 - Diagrama motivațională *(pag. 16)*

Figura 7 - Tabel de activități BMPN *(pag. 20)*

Figura 8 - *Diagrama fluxului de date DFD (pag. 22)*

Figura 9 - Captură de ecran cu un email trimis automat *(pag. 24)*

Figura 10 - Tabel cu analiza demersului de testare

Introducere

Generația mea a avut privilegiul de a crește undeva la granița dintre două lumi. Fiind născuți la începutul anilor 90, am fost ultimii care au reușit să se dezvolte într-un mediu necontaminat digital, în care toate interacțiunile se bazau pe mijloace oarecum tradiționale. La scurt timp însă, odată cu expansiunea rapidă a erei digitale, ne-am văzut nevoiți să integrăm foarte rapid în viața noastră diverse tehnologii care au schimbat radical societatea (Harari, 2015). Fiind martorii acestei tranziții am avut o perspectivă unică asupra avantajelor și dezavantajelor din ambele lumi: cea pre-digitală în care evoluția lucrurilor era mai lentă dar predictibilă și *minunata lume nouă*, hiper-conectată, în care tehnologia, societatea, și economia se dezvoltă în galop lăsând foarte puțin spațiu pentru analiză, reglementare și introspecție.

Trăind această experiență pe propria piele, generația mea a reușit să conștientizeze mult mai bine impactul pe care dezvoltarea tehnologiei îl are în diverse domenii: inclusiv cel al economiei și psihologiei. La confluența acestor ramuri ale științei s-a născut economia comportamentala, un domeniul interdisciplinar care ne demonstrează cu date concrete că deciziile sunt puternic influențate de diverși factori emoționali, biasări cognitive și diferite limitări în procesarea informațiilor (Kahneman, 2011). Citind o parte din studiile/experimentele unor autori precum Richard Thaler, Dan Ariely sau autorul mai sus citat, care au revoluționat economia, am înțeles că modul în care ne raportăm la lumea înconjurătoare trebuie gestionat cu mare atenție.

Acestea fiind spuse, contextul informatizării accelerate, în care tehnologia joacă un rol atât de pregnant, ne obligă să ne optimizăm activitatea prin soluții digitale și să oferim servicii cât mai accesibile și eficiente clienților. Din acest motiv, pentru a rezista într-o piața hiper-concurențială, în care serviciile psihologice au devenit mult mai accesibile (fiind disponibile și sub forma on-line) , am luat hotărârea de a dezvolta afacerea soției mele prin dezvoltarea unei aplicații web care să permită automatizarea unor sarcini administrative precum gestionarea programărilor, aplicarea unor teste online și stocarea datelor într-o bază de date, îmbunătățind astfel atât experiența clienților cât și a administratorului.

Obiectivul principal derivat din această evoluție accelerată a mediului economic / tehnologic ar fi dezvoltarea unei platforme web moderne, aspectuase și funcționale care să răspundă exigențelor pieței și nevoilor unui cabinet psihologic. Pornind de la conturarea unei interfețe (*front-end*) prietenoase și intuitive și până la oferirea unor funcționalități interactive (teste și

programări online) am încercat să exploatăm potențialul unor tehnologii care au trecut cu brio testul timpului și care reprezintă pilonii tehnologiei informației din prezent. Printre aceste tehnologii se numără: HTML, CSS și JavaScript (pentru construcția unei interfețe practice și funcționale), PHP și MySQL (pentru gestionarea logicii din back-end și a bazei de date unde stocam informațiile relevante), respectiv XAMPP și VisualStudio ca mediu de dezvoltare-vizualizare-testare pentru proiect.

În ceea ce privește structura lucrării vom încerca să detaliem/ilustrăm toate etapele dezvoltării aplicației web, pornind de la etapa introductivă, în care punctăm elementele ce țin de motivație, obiective, metodologie, contextul tehnologic și economic. În continuare vom dezvolta rând pe rând câteva capitole destinate metodologiei alese, tehnologiilor folosite, proiectarea aplicației și testarea ei. Pe lângă acestea, vom puncta și/sau ilustra diferite detalii tehnice care fac parte integranta a acestei lucrări. În plus, pentru a facilita înțelegerea anumitor aspecte, vom include la nevoie și diferite diagrame sau chiar fragmente de cod pentru a scoate în evidență aspectele de noutate incluse.

În concluzie, reflectând nevoia reală de digitalizare a serviciilor prestate de un cabinet psihologic prin oferirea unei soluții practice, lucrarea de față surprinde procesul de digitalizare/automatizare a unor activități administrative care ocupă un timp semnificativ din agenda unui psiholog care își gestionează singur activitatea. Dezvoltând astfel o aplicație web care să gestioneze și eficientizeze o parte din activități, atât utilizatorii cât și administratorii platformei au de câștigat. La finalul acestui proces de dezvoltare am obținut un site web complet funcțional, capabil să automatizeze etapele de programare și testare online, crescând semnificativ accesibilitatea serviciilor psihologice prestate de beneficiar. În ilustrația de pe pagina următoare putem observa mai concret funcționalitățile vizate de către acest demers de dezvoltare software.

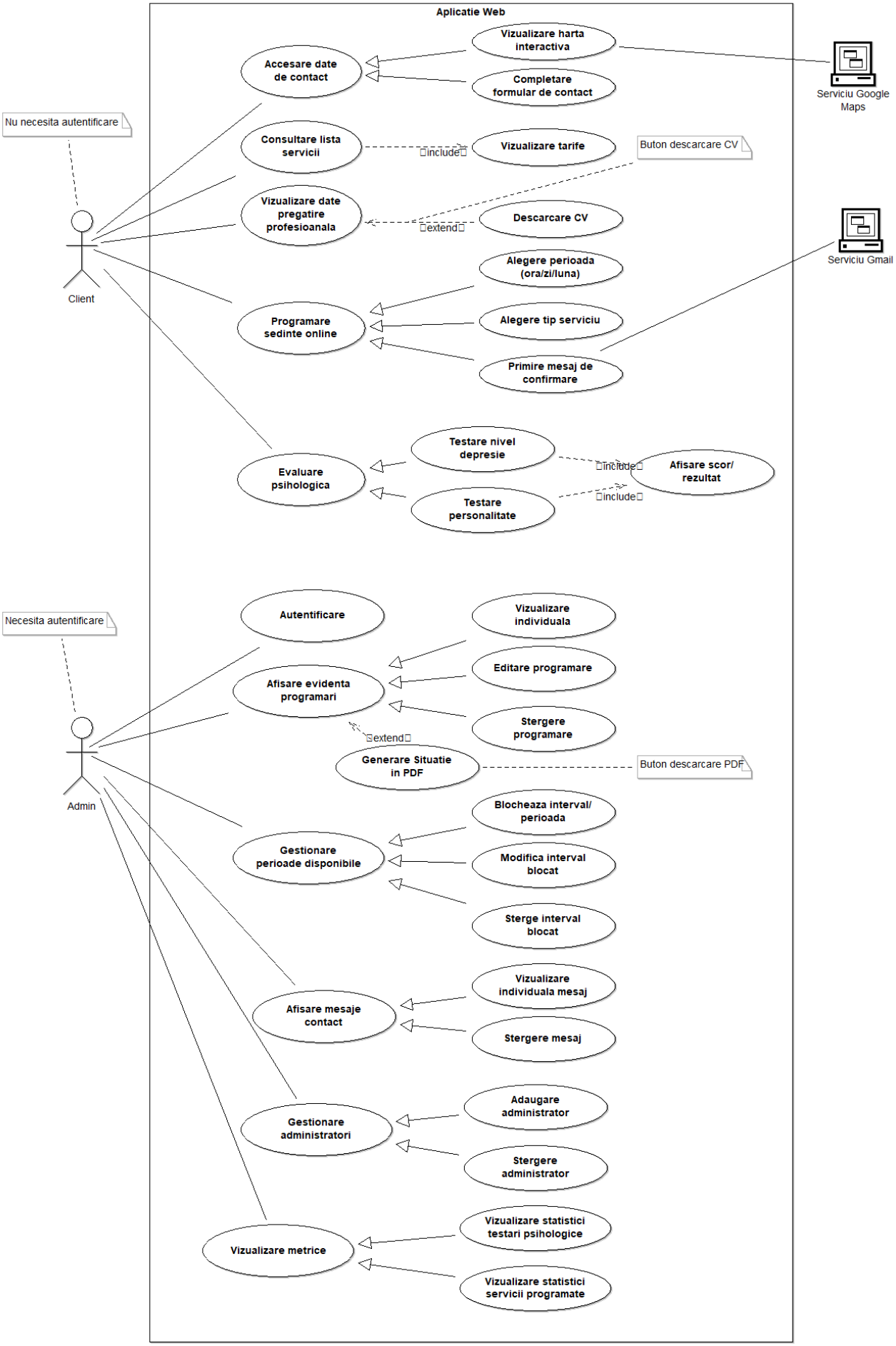


Figura 1: Diagrama use-case

1. Metodologia

1.1. Aspecte generale de metodologie

Această lucrare se încadrează în tipul A – Proiectare și implementare de aplicații web, motiv pentru care vom începe prin a detalia principalele etapele de lucru pe care am le-am parcurs în cadrul dezvoltării aplicației web.

Pentru a evidenția impactul scontat al implementării, vom debuta aceasta lucrare prin prezentarea a două diagrame care ilustrează situația actuală și cea post-implementare în care aplicația propusa va prelua o parte semnificativă din *workload*-ul psihologului.

După cum putem observa în prima ilustrație și anume *diagrama de activități* (before), prezentată mai jos, psihologului îi revin o mulțime de activități/sarcini care prezintă un potențial mare de digitalizare. Și deși aceste opțiuni sunt prezente la majoritatea clinicilor mari precum Regina Maria sau MedLife, în cazul cabinetelor individuale de psihologie, rareori găsim funcționalități mai avansate. Cele mai multe dintre ele se limitează la afișarea unor simple conținuturi informaționale: de regulă prezentarea serviciilor, a tarifelor și eventual un formular simplu de contact. În cazul nostru, putem identifica cel puțin 3-4 activități care ar putea fi executate automat de către o aplicație web: programarea ședințelor, transmiterea informațiilor utile, testarea psihologică inițială, confirmarea ședințelor și punerea la dispoziție a unor resurse utile în procesul terapeutic.

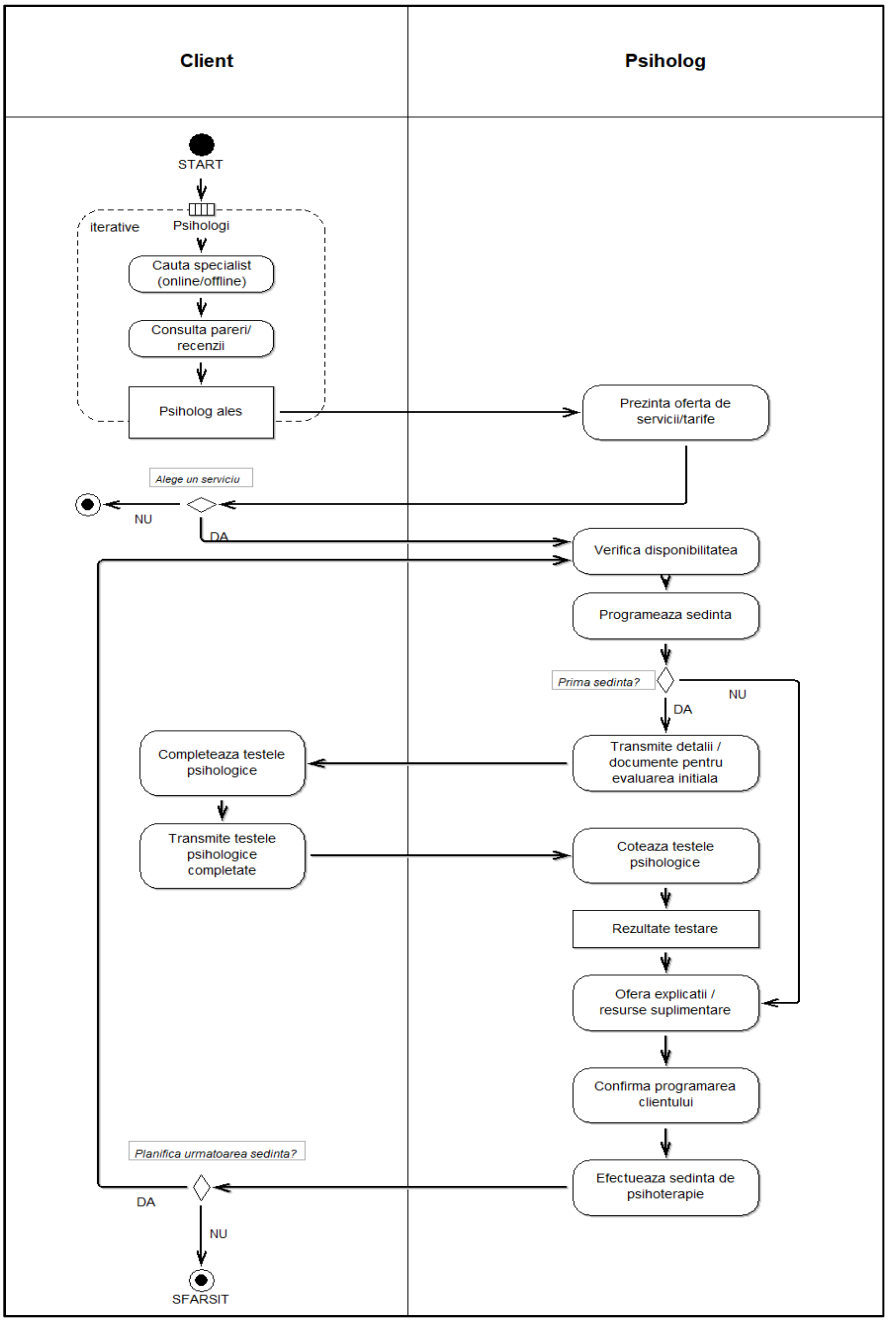


Figura 2: Diagrama de activități (înainte de implemetare)

Având ca punct de plecare aceste constatări și presați de situația economică/socială generată, am ales să urmez o abordare metodologică *orientată obiect* pentru a-mi permite să construiesc pas cu pas funcționalitățile vizate. După cum se poate observa în ilustrația de mai jos, odată pusă în practică această metodologie, aplicația web finalizată ar trebui să preia o parte semnificativă din atribuțiile de natură administrativă ale specialistului.

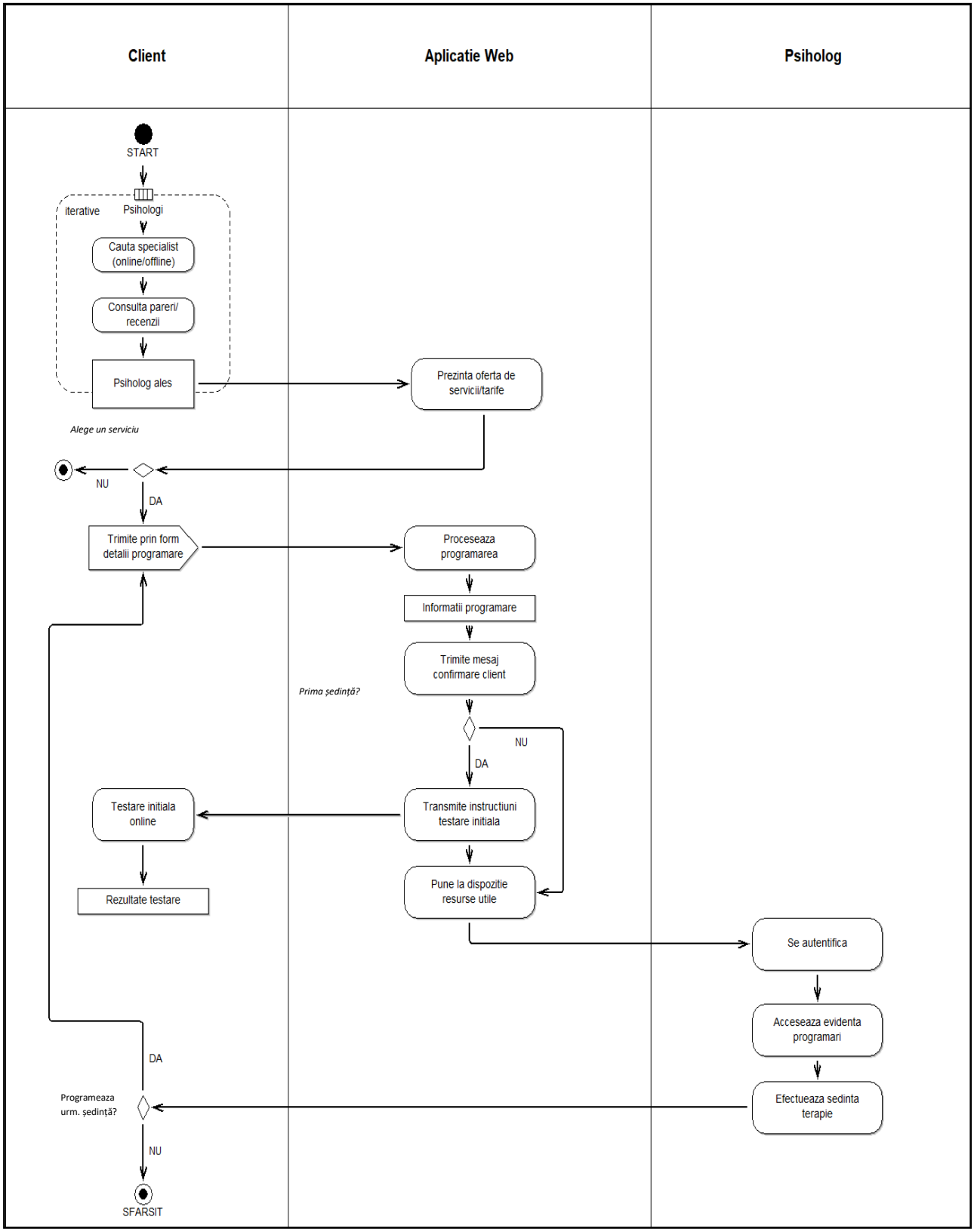


Figura 3: Diagrama de activități (după implemetare)

Analiză comparativă a celor 2 scenarii (before&after) scoate în evidență necesitatea optimizării/digitalizării activității cabinetului; lucru care ar îmbunătății considerabil atât experiența clienților, oferindu-le un acces facil la serviciile beneficiarului, cât și a administratorului/psihologului care ar putea să își investească timpul câștigat în pregătirea lui în alte activități ce țin de actul terapeutic.

După cum se poate vedea în tabelul de mai jos, există cel puțin cinci aspect vizate care ar urma să fie semnificativ îmbunătățite ca urmare a implementării acestei aplicații web.

Nr. crt.	Aspect vizat	Înainte de implementare	După implementare
1.	Gestionarea programărilor	Programările se fac prin telefon / mesaje, lucru care poate duce la posibile suprapuneri (ele fiind scrise manual într-o agendă).	Programarea online, pe un calendar digital, cu vizibilitate clară a intervalelor disponibile, care nu permite suprapunerea de programări.
2.	Accesibilitatea serviciilor	Informațiile privind serviciile sunt disponibile doar prin telefon, în limita unui program de lucru.	Serviciile sunt prezentate, detaliate și disponibile permanent pe platforma web.
3.	Aplicarea testelor psihologice	Procesul necesită o muncă mai laborioasă de administrare și cotare manuală.	Testarea se va putea efectua online, rezultatul și interpretarea fiind oferite în timp real imediat dupa finalizarea testului.
4.	Experiența utilizatorului	Pentru orice solicitare este necesar un contact direct cu psihologul.	Majoritatea informațiilor și a serviciilor devin accesibile la doar câteva click-uri distanță.
5.	Exprienţa administratorului	Informațiile de natură administrativă sunt trecute pe agenda, olograf, fiind costisitoare ca timp și greu de coroborat și analizat ulterior.	Toate datele ce țin de programări, teste online, mesaje primite se stochează într-o baza de date și pot fi vizualizate/analizate cu ușurință.

Figura 4: Tabel analiză comparativă înainte&după implementare

Pornind din acest punct zero fără o experiență profesională anterioară în domeniul IT dar cu un stil de lucru foarte disciplinat, am considerat că cel mai bine ar fi să îmi organizez procesul de lucru în concordanță cu filosofia modelului *Agile* (Beck et al., 2001). Această abordare metodologică care permite o dezvoltare iterativă și flexibilă a reprezentat unul dintre atuurile principale care m-a determinat să aleg stilul Agile, întrucât acesta s-a mulat perfect pe modul meu de gândire. Lucrând astfel într-un mod consecvent, regulat am reușit să abordez

problemele mai complexe prin divizarea lor în sarcini mai mici și ușor de gestionat. Totodată, fiind absolvent unei facultăți de psihologie (atât eu cât și soția mea), am încercat să colaborăm cât mai mult pentru a alinia constant obiectivele de dezvoltare a acestei aplicații web, în așa fel încât să putem beneficia într-un mod real de pe urma materializării ei.

Ca și model de programare, am ales abordarea orientată obiect, lucru care s-a dovedit a fi o alegere foarte convenabilă pentru gestionarea unei aplicații destul de complexe pentru un începător. Spre deosebire la arhitectura MVC, modelul orientat obiect mi-a permis să sparg codul în mai multe module, cu o structura clară a obiectelor/claselor și a relațiilor dintre ele. Această abordare a sporit accesibilitatea și întreținerea/modificarea codului, facilitând extinderea progresivă a funcționalităților fără a afecta aplicația ca întreg.

1.2. Tehnologii specifice

Pentru realizarea platformei web am utilizat o serie de tehnologii modern care mi-au permis să creez atât o interfață atractivă cât și o funcționalitate optima pentru nevoile cabinetului de psihologie. În continuare vom enumera principalele tehnologii utilizate și rolul fiecăreia în cadrul aplicației:

1. HTML (HyperText Markup Language) – Codul scris în acest limbaj reprezintă practic scheletul aplicației web; el fiind utilizat pentru a organiza și afișa elementele de conținut pe pagină: de la secțiunile principale ale site-ului și până la cele mai mici elemente afișate (iconițe, formulare, imagini, titluri, butoane etc.)

2. CSS (Cascading Style Sheet) – Cu ajutorul acestui limbaj de stilizare am personalizat și înfrumusețat aspectul elementelor construite folosind tehnologia HTML, oferind astfel un design modern și minimalist interfeței vizuale. Fiind conștient de impactul primului contact cu un produs/serviciu nou (Ariely, 2008), am încercat să exploatez la maxim capabilitățile acestei tehnologii în încercarea de a crea o experiență de navigare intuitivă, responsivă și atractivă din punct de vedere vizual (Freeman & Robson, 2014).

3. JavaScript a jucat un rol esențial în cadrul aplicației prin modul în care m-a ajutat să creez unele comportamente mai avansate pentru validarea formularelor, gestionarea unor animații și mai ales pentru afișarea dinamică a secțiunii de programări / teste online, unde există o puternică interdependență între front-end și back-end.

4. PHP (Hypertext Preprocessor) – a fost folosit pentru a scrie logica de back-end, contribuind astfel la procesarea și transferul datelor de la formulare către baza de date și viceversa; totodată, folosind această tehnologie am reușit creez un mediu în care activitățile administrative sunt automatizate în mare măsură scăzând semnificativ volumul de muncă (extra-profesională) pentru beneficiar.

5. MySQL – A contribuit la stocarea datelor vehiculate în cadrul platformei. Mai mult, prin diverse interogări SQL, aceste informații pot fi preluate și afișate în timp real pe interfața vizuală a front-end-ului.

Acestea fiind spuse, folosind acest instrumentar, am reușit să construiesc un site web bine structurat, cu care îmbină un design atractiv cu multiple funcționalități complexe, contribuind astfel la o experiență de utilizare fluidă. Fiecare din tehnologiile menționate mai sus a adus un aport specific la atingerea acestor obiective: HTML și CSS au asigurat o prezentare clară și modernă, JavaScript a integrat partea de comportament dinamic, PHP a articulat logica de scripting a back-end-ului, iar MySQL.

După finalizarea celor două componente majore ale aplicației, am început un proces mai lung de testare/ajustare a diferitor parametrii pentru a mă asigura că interacțiunea dintre utilizator și platformă este una cât mai fluidă și intuiti vă. Aici am pus accentul pe stilizarea și adaptarea conținutului pe diverse tipuri de display (folosind media query).

În consecință, pentru a îndeplini cerințele *beneficiarului*, toate aceste tehnologii trebuie să lucreze într-o manieră eficientă, marcată de o puternică interdependentă între subsistemele aplicației. Diagrama de mai jos ilustrează arhitectura de bază a aplicației web și prezintă modul de comunicare a pilonilor de bază: front-end, back-end, baza de date și serviciile oferite de serverele externe Google.

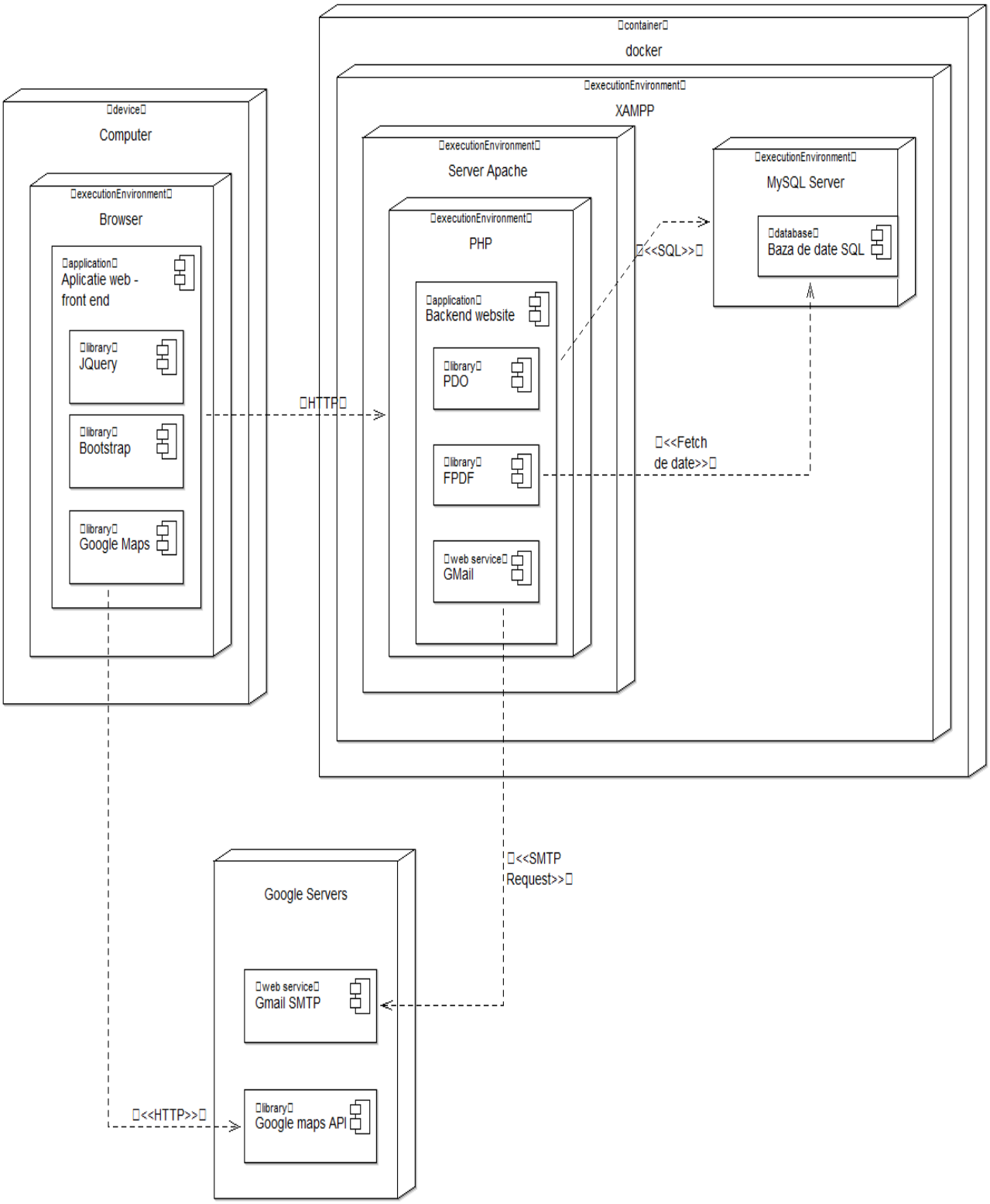


Figura 3: Diagrama de deployment

2. Analiză și proiectare

2.1. Analiza contextului de business

Modul în care ne raportăm la toate serviciile de pe piață s-a schimbat radical în ultimul deceniu. Accesibilitatea device-urilor inteligente, aria de acoperire și costurile reduse ale unui abonament de internet au creat premisele unei dezvoltări foarte accelerate mai ales în ceea ce privește soluțiile software. Profitând de acest context prielnic, majoritatea prestatorilor de servicii medicale/psihologice au căutat să exploateze potențialul enorm al acestui trend ireversibil. Catalizând și mai mult acest fenomen, pandemia de COVID-19 au forțat angajatorii să adopte rapid soluții digitale pentru a-și păstra personalul și clienții.

Devenind astfel o prioritate strategică, guvernul a acordat sectorului IT o scutire de la plata impozitului pe venit. Având acest avantaj competitiv, companiile localizate în România (și mai ales în Cluj-Napoca) au contribuit semnificativ la bunăstarea economică a regiunii. Totuși, chiar dacă Ministerul Sănătății a investit semnificativ în digitalizarea sistemului de sănătate, cabinetele de psihologie s-au dezvoltat disproporțional, fiind concentrate mai ales în marile zonele urbane. Din acest punct de vedere, măsurile de distanțare/izolare impuse de situația epidemiologică, i-au determinat pe psihologi (și nu numai) să își presteze și în mediul online serviciile. Confruntându-se brusc cu aceste limitări, piața s-a adaptat rapid și în doar câțiva ani, locurile de muncă *remote* au devenit omniprezente pe platformele de recrutare.

Un exemplu reprezentativ în domeniul psihologiei informatizate îl reprezintă compania clujeana Cognitrom, care creat una dintre primele platforme online dedicate psihologilor și terapeuților. Aceasta inițiativă timpurie materializată în anul 2019 a demonstrat beneficiile digitalizării în domeniul psihologiei și a creat un precedent pentru informatizarea tehnologiei IT în sfera serviciilor de sănătate mentală și nu numai; aceste informații fiind puse la dispoziția publicului de G4Media.ro. (2020, martie 16).

Facem un salt în timp de doar câțiva ani, și astăzi, majoritatea clienților se așteaptă ca serviciile psihologice să fie accesibile online, fie sub forma ședințelor de terapie desfășurate online fie sub prin servicii de evaluare aplicată online. În acest context, dezvoltarea unui site web care să integreze cel puțin o parte din cerințele actuale ale pieței nu mai reprezintă un lux, ci o necesitate strategică de supraviețuire/dezvoltare economică.

2.2. Motivația lucrării:

Decizia de a crea o platformă web pentru cabinetul de psihologie al soției mele este determinată de un cumul de factori, printre care se regăsesc:

1) **Tendințele pieței în era digitală**: pe măsură ce incidența problemelor de natura psihologică crește, afecțiunile mentale devin un subiect tot mai ușor de abordat în societatea noastră. Din acest motiv, cabinetele care încorporează soluții digitale beneficiază de un avantaj competitiv semnificativ (Global Market Insights, 2023).
2) **Profesionalismul interfeței de promovare/utilizare:** un site web bine structurat, cu funcționalități dinamice integrate, oglindește într-o anumită măsură profesionalismul psihologului și poate inspira un plus de încredere potențialilor clienți.
3) **Creșterea accesibilității serviciilor psihologice**: o platformă online permite clienților să programeze o ședință cu ușurință, sa acceseze informații relevante și să beneficieze de servicii precum evaluări online la distanță (Kesrouani, 2023)

Pentru a ilustra într-un mod mai complex aspectele ce țin de aspectele de motivația acestui demers de dezvoltare software am conceput diagrama prezentată mai jos. Respectând structura de bază a diagramelor motivaționale am punctat mai jos, sub formă schematică elementele ce țin de factori motivatori (dorințe și nevoi ale beneficiarilor), obiective SMART, rezultate așteptate (cu indicatori cât se poate de cuantificabili), diferite cerințe și constrângeri care se vor transpune implicit în cerințelele software-ul. Mai mult, fiind derivate din aceste cerințe, serviciile/funcționalități aplicației Web sunt și ele enumerate și clasificate în funcție de cerințele beneficiarului pe care încearcă să le îndeplinească.

În plus, făcând parte integrantă a acestei diagrame, sunt prezentate și alte variante alternative prin care se pot atinge obiectivele/rezultatele dorite. Aici, demersurile depășesc sfera dezvoltării software și abordează tehnici din domeniul marketing-ului și a tehnicilor de promovare.

În concluzie, fiind una dintre profesiile unde activitatea nu necesită o prezență fizică obligatorie și luând în considerare răspândirea ariei de disponibilitate a serviciilor de internet (atât ca aria de acoperire cât și costul redus) am considerat ca digitalizarea activităților pretabile mediului online reprezintă o cerință obligatorie dacă ne raportăm la evoluția economiei moderne care taxează aspru business-urile care nu se adaptează rapid la economia de piață în contextul digitalizării accelerate (Moudatsou, 2024).

2.3. Cerințele și beneficiile implementării:

În ceea ce privește cerințele de bază ale platformei web create, am pornit cu următoarele obiective:

- ✓ Interfața să fie cât mai prietenoasă și intuitivă, astfel încât experiența de utilizare să fie cât mai facilă atât pentru clienți cât și pentru administrator/psiholog.
- ✓ Sarcinile administrative să fie cât mai mult automatizate / digitalizate pentru a reduce sarcinile de natură administrative
- ✓ Datele care circulă prin aplicație să fie stocate în baze de date sigure
- ✓ Platforma web să fie compatibilă atât cu sistemele desktop cât și cu dispozitivele mobile

Atingerea acestor obiective/cerințe ar crea o serie de benificii pentru toate categoriile de utilizatori care ar folosi aplicația. Câteva dintre cele mai importante ar fi:

Practic, prin acest demers se dorește atingerea următoarelor ținte strategice în ceea ce privește evoluția pe termen lung a cabinetul de psihologie:

- ✓ Eliminarea greșelilor umane la procesul de programare (ex. suprapunerea clienților), ca urmare a automatizării procesului. Implicit, timpul alocat acestei sarcini administrative ar crea
- ✓ Eficientizarea procesului de programare online și/sau testare psihologică, clienții având posibilitatea de a efectua aceste operațiuni fără a fi nevoie de o interacțiune directă cu psihologul.
- ✓ Promovarea cabinetului și atragerea de noi clienți prin creșterea vizibilității și în mediul online. Deși majoritatea pacienților vin pe bază de recomandare, într-o piață hipercompetitivă precum cea a serviciilor medicale, orice mic avantaj poate face diferența.

Pornind de la aceste cerințe și beneficii scontate, consider că digitalizarea proceselor administrative ale unui cabinet de psihologie reprezintă o soluție eficientă și necesară pentru a optimizarea activității și facilitarea accesului la servicii psihologice. În contextul actual, în care Cognitrom și alte initiative locale au demonstrat potențialul tehnologiei informației în domeniul psihologiei, proiectarea unei platform web pentru un cabinet de psihologice devine o decizie strategică esențială pentru supraviețuirea pe piață. O astfel de inițiativă nu doar că răspunde

cerințelor pieței actuale, dar și poziționează cabinetul într-un poziție facilă, fiind pregătit pentru provocările și oportunitățile erei digitale.

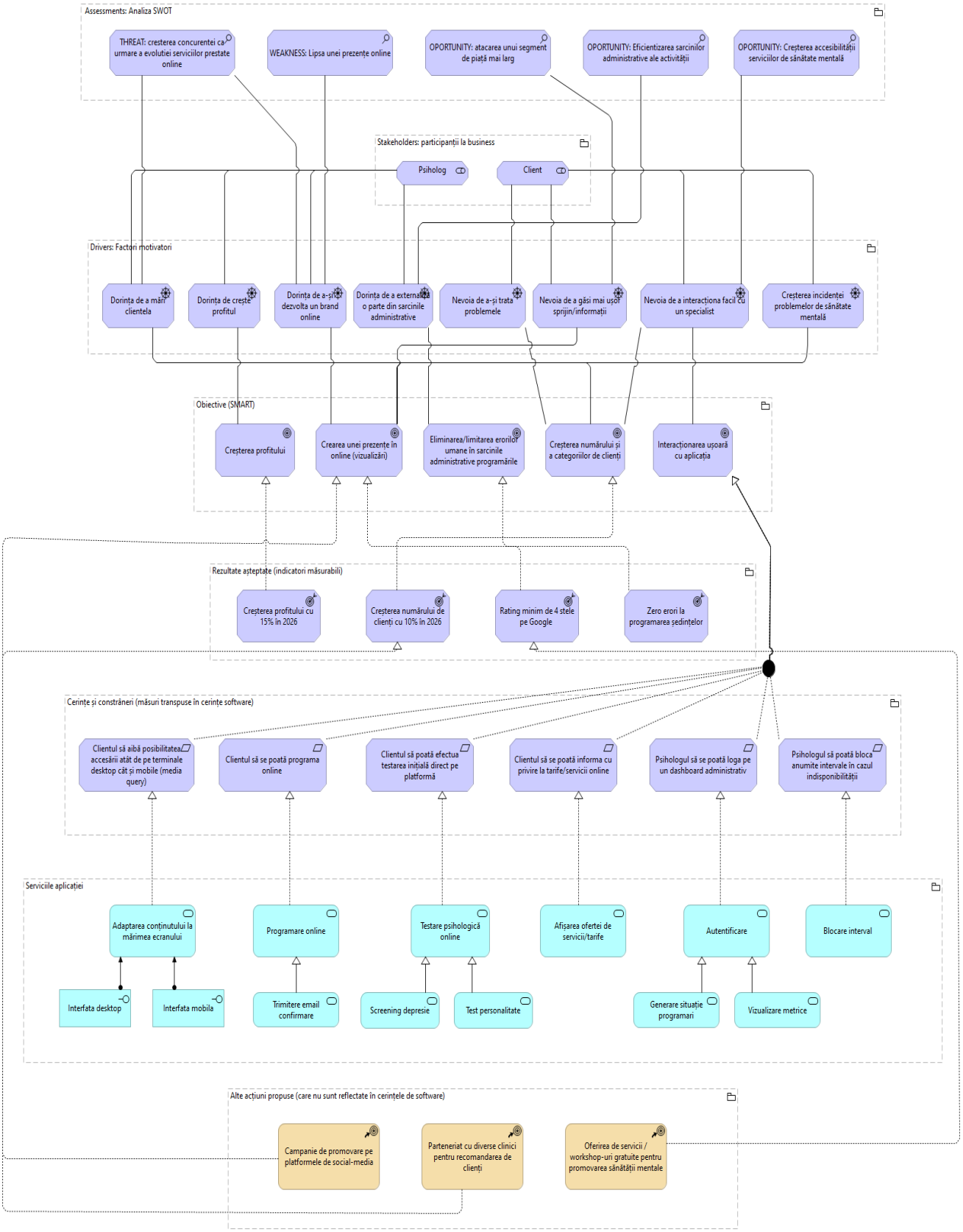


Figura 6: Diagrama motivațională

2.4. Elemente de proiectare a aplicației web

2.4.1. Considerații generale privind proiectarea software

De cele mai multe ori, faza incipientă a dezvoltării unui sistem informatica debutează cu o analiză a cerințelor programului, care sunt stabilite de beneficiar în strânsă colaborare cu programatorul. Ideea acestui demers este de a contura un plan cât se poate de coerent pentru o implementare ulterioară. În această etapă ideile se transformă încet în lucruri concrete, iar funcționalitățile discutate inițial din punct de vedere conceptual se transformă încet în structuri logice și fluxuri/procese clare și secvențiale. Fără o proiectare responsabilă, există riscul de a dezvolta sisteme neintuitive pentru vizitatori și greu de întreținut pentru administratori. Tocmai din acest motiv am ales o abordare Agile, orientată obiect (McFarland, 2015).

În cazul de față, scopul a fost crearea unei aplicații web care să sprijine activitatea unui cabinet psihologic prin digitalizarea unor activități administrative cu potențial mare de digitalizare. Un accent mai mare a fost pus pe două funcționalități care să ofere și un avantaj competitiv pe piața locală, și anume: programarea ședințelor și testarea psihologică online. Totodată, pentru a elimina dependența psihologului față de programator, am investit mult efort în crearea unei interfețe de administrare (dashboard) prin care specialistul să poată vizualiza/gestiona programările, intervalele disponibile și diverse statistici/evidențe utile în activitatea lui de zi cu zi. Toate acestea sunt condiționate de un proces de autentificare cu credențiale.

Pentru a ilustra aceste obiective funcționale și/sau logice am apelat la modele vizuale standarziate, care mi-au permis să structurez coregrafia funcțională a a aplicației înainte de a trece la scrierea propriu-zisă a codului. Aceste modele internaționale au avantajul că ne oferă o viziune de perspectivă și contribuie la o identificare timpurie a eventualelor lacune de de logică sau arhitectură încă din faza de concepție a aplicației.

2.4.2. Modelare conform standardelor UML si BPMN

Folosind cu preponderență standardul UML (Unified Modeling Language), care este probabil cel mai răspândit limbaj de modelare software, am încercat reprezint cât mai clar structura internă a sistemului informatic și comportamentul acestuia. În esență, scopul și rolul acestor standarde internaționale este de a ajuta la „traducerea” schematică a procesului de dezvoltare în așa fel încât să fie înțeles atât de programatori cât și de publicul larg (beneficiari/utilizatori ai soft-urilor). Acest atuu al versatilității m-a determinat să aleg UML ca principal tool pentru a abstractizarea demersurilor de dezvoltare a aplicației.

În primul rând, folosind instrumentele puse la dispoziție ale standardului, am utilizat program Bee Up pentru a creiona o diagrama de cazuri de utilizare (*use case diagram*) pentru a oferi o imagine de ansamblu asupra interacțiunilor dintre utilizatori / administrator și aplicație. Mai concret, am stabilit două roluri principale (client și administrator) și am extins principalele funcționalități la care poate apela fiecare. Aici, accentul nu a fost pus pe proces de interacțiune ci pe enumerarea tuturor funcționalităților aplicației și pe modul în care ele sunt accesibile diverșilor utilizatori: în cazul nostru clienți obișnuiți sau administratori ai aplicației. Clientul are acces la funcționalitățile de testare online (cu vizualizarea rezultatului), programare online și trimiterea unui mesaj prin intermediul formularului de contact. De cealaltă parte, administratorului (adică psihologul) are acces, după logarea cu credențiale, la un dashboard în care poate accesa situația programărilor existente, poate bloca anumite intervale de timp, poate vizualiza statistici/metrice. Aceste interacțiuni se găsesc ilustrate și în diagrama use-case unde evidențiem actorii și aciunile concrete la care au acces în cadrul platformei.

Totodată, pentru a înțelege mai bine dinamica acestor interacțiuni, am realizat mai multe diagrame, printre care și două diagrame de activitate (before&after). Acestea descriu pas cu pas comportamentul sistemului în funcție de deciziile utilizatorilor. Prin compararea lor side-by-side, putem observa cum aplicația software realizată automatizează mare parte din procesele uzuale, reducând timpul de răspuns și eliminând eventualele erori umane. În plus, atuul principal al acestor facilități digitale este că simplifică foarte mult comunicarea dintre client și psiholog.

Un alt instrument folosit în cadrul procesului de proiectare a fost diagrama de flux de date prin care am descris informațional între componentele sistemului: interfața grafică (GUI), serverul aplicație și baza de date. Practic, modelul creionat arată cum datele provenite din formularele de pe aplicație (programări, rezultate teste, perioade blocate, etc.) sunt colectate, procesate și afișate. Această diagramă m-a ajutat mult în etapa de proiectare a bazei de date și mi-a permis să înțeleg cum vor interacționa între ele modulele aplicației.

Completând imaginea sistemului din punct de vedere al infrastructurii am inclus în această lucrare și o diagramă de implementare. Aceasta diagramă evidențiază faptul că aplicația web a fost gândită pentru a funcționa într-un regim clasic, pe modelul client-server, fiind totodată pregătită pentru eventuale extinderi ulterioare și/sau pentru integrarea altor servicii externe. Am ales această abordare orientată-obiect pentru a putea lucra pas-cu-pas, iterativ, după metodologia Agile (Turk D. 2014). Fără a avea vreo experiență anterioră în programare, am

considerat că această abordare este cea mai potrivită/accesibilă. Revenind la aspectele tehnice ale diagramei de implementare, în cadrul ei sunt ilustrate: serverul web (care rulează local prin XAMPP), baza de date MySQL și dispozitivele de pe care utilizatorii pot accesa aplicația. Altfel spus, această diagramă pune în scenă infrastructura de bază.

Pe lângă acestea, am considerat util să realizez și o diagrama motivațională, inspirându-mă din modelele care abordează procesele de arhitectură într-o manieră mai accesibilă pentru non-programatori. Rolul acesteia este de a justifica dezvoltarea aplicației prin corespondența dintre obiectivele de business (obiective SMART) și diversele aspecte ce țin de funcționalitatea ei tehnică. Un exemplu concret ar fi dorința/obiectivul de a reduce timpul și erorile care au loc în procesul de programare a ședințelor, care s-a transformat într-o funcționalitate concretă: sistemul de programare online. În același mod, nevoia de a simplifica procesul de testare online a fost îndeplinită prin crearea unui portal de testare online unde clienții pot să-și auto-administreze 2 tipuri de teste psihologice: screening-ul pentru depresie și un test de personalitate. Într-un mod similar, această diagramă scoate în evidență și nevoile/obiectivele psihologului, care țin în mare măsură de eficientizarea/digitalizarea proceselor administrative se transpun în module de funcționalitate distincte.

În cele din urmă, deși am folosint preponderent standardul UML, am ținut să integrez în cadrul lucrării și elemente specifice standardului BPMN (Business Process Model and Notation), pentru a ilustra/descrie procesele administrative într-o manieră cât mai accesibilă celor fără competențe digitale să zicem. Acest standard foarte des folosit se pretează foarte bine în modelarea secvențelor de acțiuni interne, fiind recomandat pentru descrierea logicii și a unui fir logic pentru întreg procesul din spatele aplicației: de la verificarea programărilor până la procesul de evaluarea inițială și accesul la informațiile utile pentru primele ședințe.

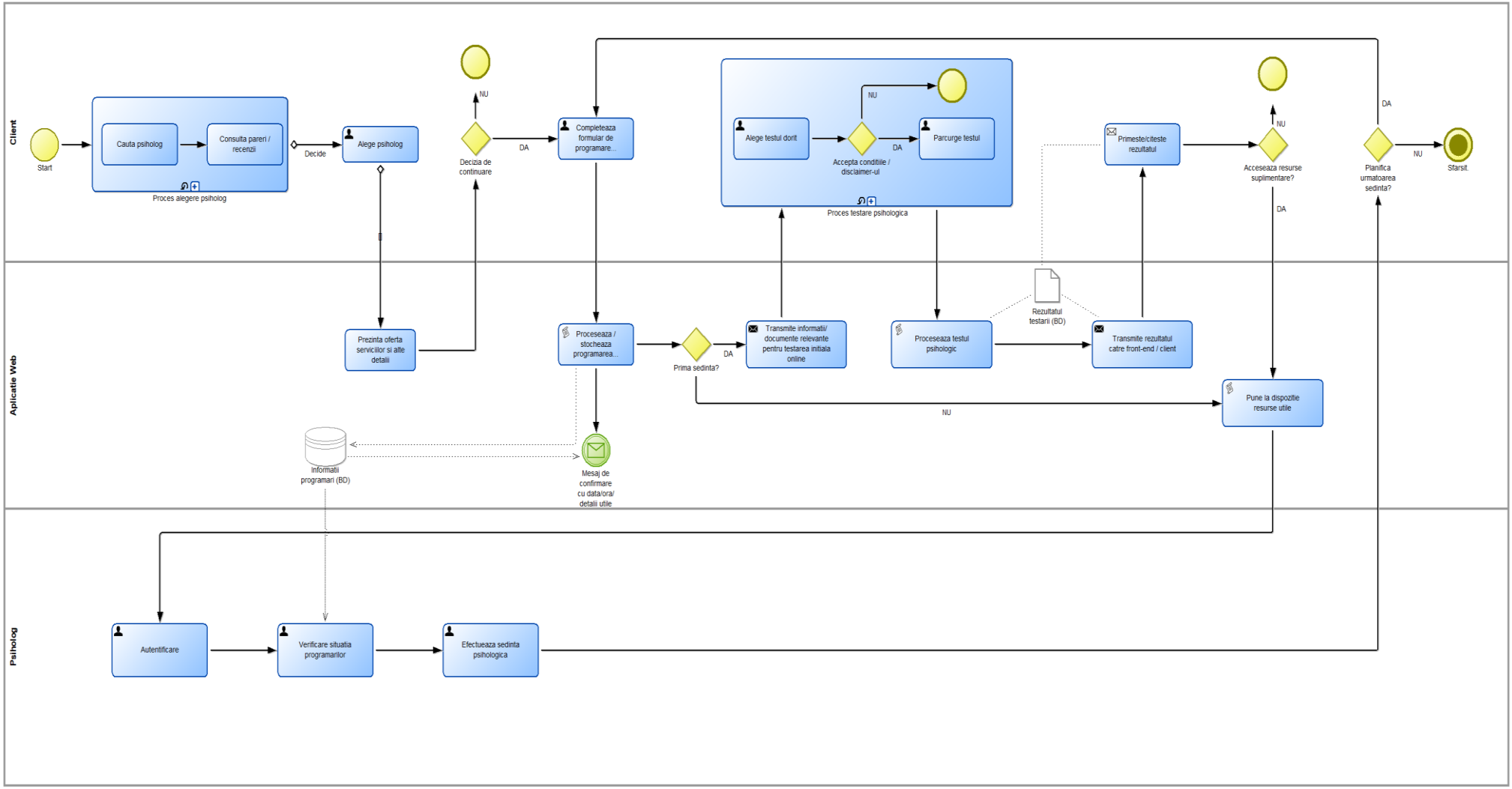


Figura 7: Diagrama de activități BPMN (after)

2.5. Fluxurile de utilizare și logica aplicației

În sensul larg al cuvântului, modelarea logicii aplicației are ca scop descrierea scenariilor posibile în care clienții (utilizatorii) vor putea interacționa cu aplicația web. Importanța acestui demers stă în identificarea situațiilor de excepție, definirea regulilor de validare și construirea unui sistem coerent și mai ales intuitiv.

Pentru a realiza acest obiectiv am început prin a stabili fluxurile de activitate pentru acțiunile de bază. Spre exemplu, pentru a realiza o programare online, utilizatorul accesează un formular, îl completează, selectează data și ora dorită. În paralel, sistemul/aplicația verifică în timp real disponibilitatea prin a popula formularul doar cu intervalele disponibile. După alegerea intervalului, programarea se salvează și se trimite un email de confirmare automat în care sunt menționate detaliile programării și un link către diverse resurse utile. Acest tip de interacțiune a fost modelată într-o diagramă de activitate detaliată și a fost implementat în cadrul aplicației.

Un alt flux important, este cel legat de evaluare psihologică. Aici, aplicația ne oferă varianta de a accesa 2 tipuri de test (unul de depresie și unul de personalitate) și ne obligă să acceptăm un consimțământ informat înainte de a accesa testul efectiv. După această formă de validare testul generează un set de întrebări și variante de răspuns, iar ulterior, după completare sistemul calculează și afișează automat rezultatul testului și un scurt buletin informativ care explică pe scurt rezultatul obținut.

Pentru fiecare astfel de proces, am respectat principiul de separare a funcționalităților, astfel ca fiecare componentă a sistemului (interfața, server, baza de date) să fie clar delimitate. Practic, interfața colectează datele din formulare, serverul procesează logica iar în baza de date sunt stocate informațiile în mod structurat și sigur.

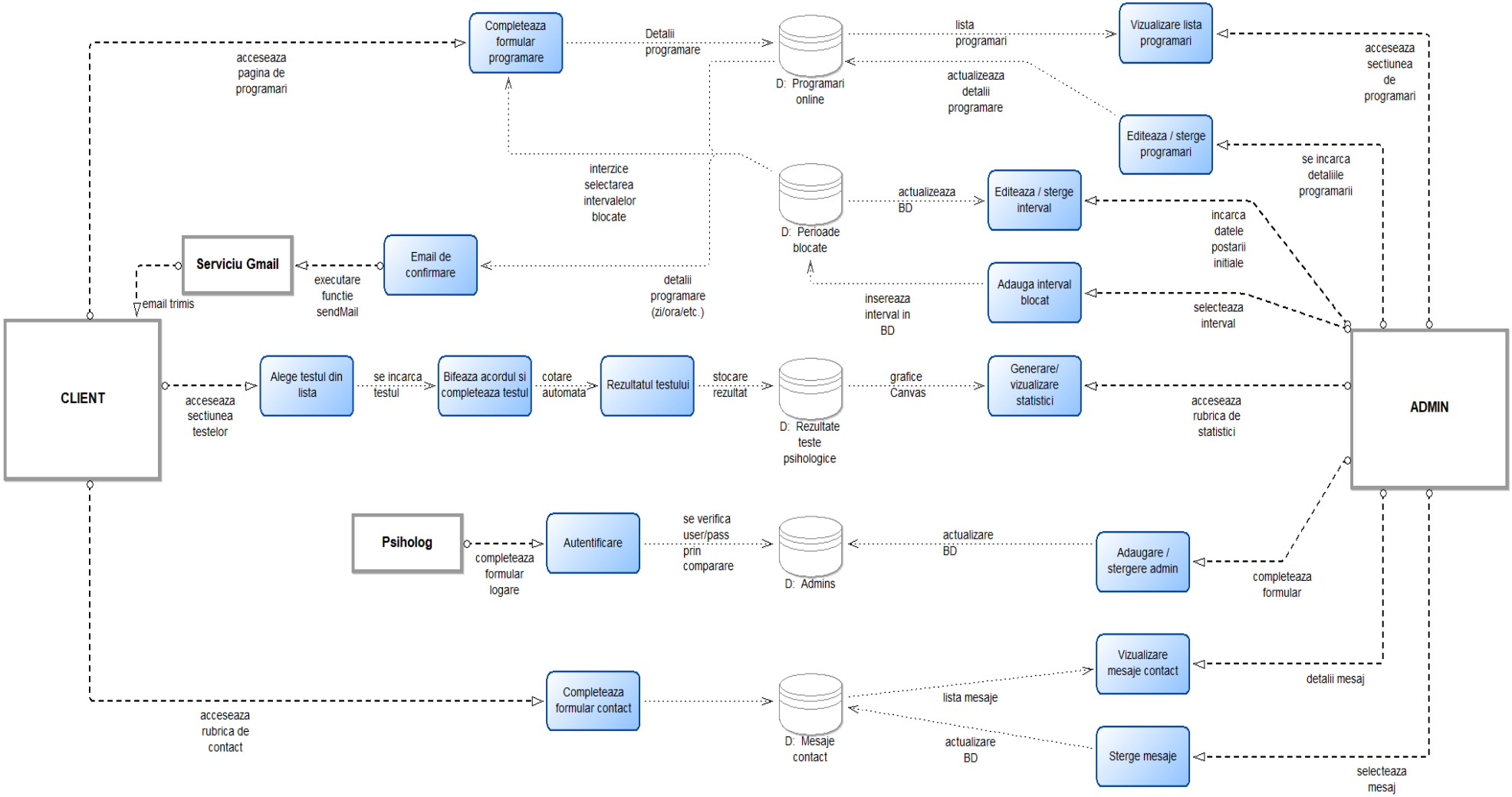


Figura 8: Diagrama fluxului de date (DFD)

2.6. Sinteză și concluzii

În concluzie, fiind baza oricărui demers de dezvoltare, proiectarea a jucat un rol foarte important în tot procesul. Pornind de la utilizarea standardelor de modelare UML și BPMN care au oferit un cadru de analiză, organizare și comunicare a fluxurilor/deciziilor am construit mai multe diagrame cu rol distinct. Fiecare astfel de diagramă a fost contribuit la clarificarea unor aspecte importante, precum: cine sunt utilizatorii; cum interacționează aceștia cu aplicația web; ce tip de informații circulă și cum sunt ele manipulate de subsisteme și cum reacționează sistemul în funcție de input-urile primite de la utilizatori/administrator.

Nefiind doar un exercițiu academic, acest demers de proiectare a fost unul cât se poate de necesar pentru a implementa o aplicație web funcțională și intuitivă. Prin modelarea riguroasă a logicii și a fluxurilor de informații am încerc să reduc riscurile de eroare și să ofer utilizatorilor o experiență plăcută de navigare. În consecință, lăsând și loc pentru eventuale extinderi ulterioare, sunt de părere că etapa proiectării reprezintă fundamentul oricărei aplicații de succes, indiferent de dimensiunea / specificul domeniului pentru care e dezvoltată. Cheia, din punctul meu de vedere, este simplitatea și eficiența.

3. Implementare

3.1. Aspecte generale ale implementării

Din punct de vedere al implementarii propriu-zise, platforma a fost dezvoltată folosind un set de tehnologii web moderne precum HTML, CSS, JavaScript pentru partea de interfață, iar PHP și MySQL pentru partea de server și gestionarea datelor. Obiectivul principal a fost construirea unei aplicații funcționale, ușor de utilizat, care să răspundă nevoilor specifice ale unui cabinet psihologic, atât din perspectiva utilizatorului final, cât și din cea a administratorului.

În dezvoltarea aplicației au fost utilizate concepte fundamentale din programare web, precum manipularea DOM, procesarea datelor pe server, validarea și transmiterea formularelor, precum și interacțiunea cu baza de date. De asemenea, au fost implementate funcționalități legate de automatizare, filtrare, protecție a conturilor, afișare dinamică a datelor și reprezentări vizuale interactive.

În continuare, voi prezenta fragmente de cod însoțite de explicații și capturi de ecran pentru a ilustra câteva dintre aspectele mai interesante și tehnice ale procesului de implementare.

3.1.1. Transmiterea automata a unui email după fiecare programare online

```
  if ($stmt->execute()) {
    $to = $email;
    $subject = 'Confirmare programare @ cabinet psihologie Pustan
Andrada';
    $message = "Bună ziua! \n Vă informăm că sunteți programat în
                data de $date la ora $time  pentru  o ședință de
                $serviceType. etc. \n
                Cu respect,\n
                Psiholog clinician & psihoterapeut\n
                Andrada Pustan";

    sendEmail($email, $subject, $message);

    echo "Programarea a fost făcută cu succes!";
  } else {
    echo "Eroare SQL: " . $stmt->error;
  }
  $stmt->close();
  $conn->close();
} else {
  echo "Cerere invalidă.";
}
function sendEmail($to_email, $subject, $body)
{
  $headers = "From: Psiholog clinician & psihoterapeut ANDRADA
PUSTAN ";
  mail($to_email, $subject, $body, $headers);
}
```

După ce interogarea SQL pentru salvarea programării în baza de date este executată cu succes, scriptul construiește un mesaj personalizat ce conține detalii despre programare.

Funcția sendEmail() este apelată pentru a expedia emailul folosind funcția nativă mail() din PHP.

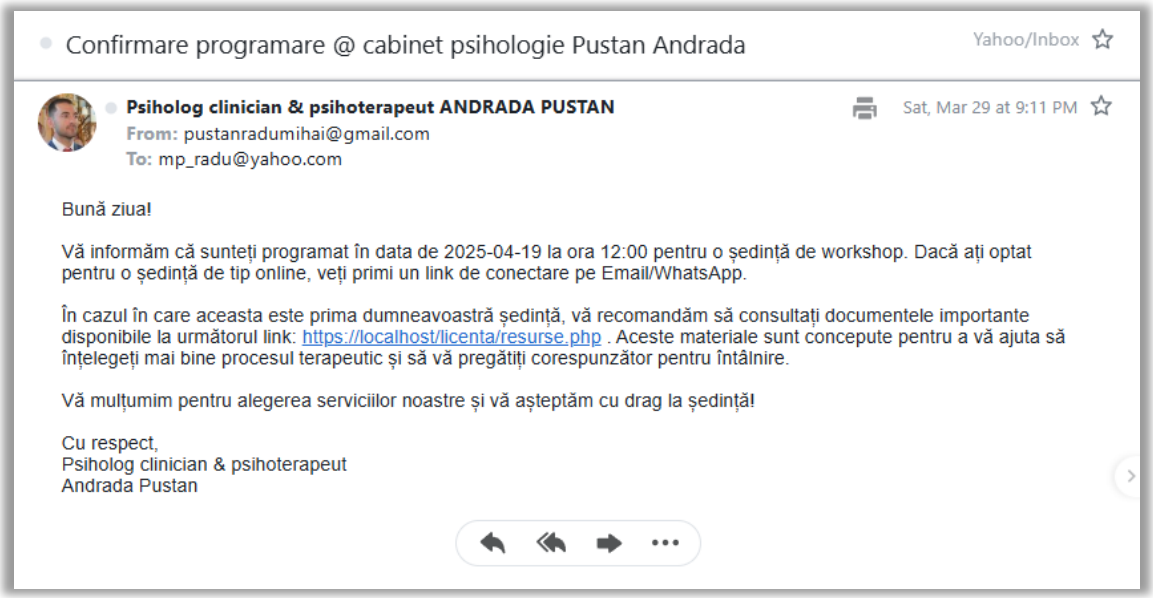


Figura 9: Captura de ecran cu exemplificarea unui email primit ca rezultat al execuției funcției

3.1.2. Formatarea unor date pentru a fi mai ușor de citit de către utilizatori.

În acest caz merită menționat faptul că majoritatea bazelor de date (inclusiv MySQL) lucrează implicit cu formatul YYYY-MM-DD. Logica din spatele acestei alegeri este ca în acest fel datele pot fi sortate cronologic mai ușor, evitând totodată și ambiguitatea/confuzia generată de faptul că nu există un sistem universal de raportare (țări precum SUA preferă formatul Y-m-d, iar țările europene optează cu preponderență către stilul d-m-Y).

```
<div class="details-container">
    <h2>Detalii Programare</h2>

    <div class="details">
      <p><strong>ID:</strong> <?php echo $row['id']; ?></p>
      <p><strong>Nume:</strong> <?php echo htmlspecialchars($row['name']); ?></p>
      <p><strong>Email:</strong> <?php echo htmlspecialchars($row['email']); ?></p>
      <p><strong>Telefon:</strong> <?php echo htmlspecialchars($row['telefon']); ?></p>
      <p><strong>Data:</strong> <?php echo date('d-m-Y', strtotime($row['date'])); ?></p>
      <p><strong>Ora:</strong> <?php echo substr($row['time'], 0, 5); ?></p>
      <p><strong>Tip Serviciu:</strong> <?php echo htmlspecialchars($row['serviceType']);
?></p>
      <p><strong>Tip Ședință:</strong> <?php echo htmlspecialchars($row['sessionType']);
?></p>
      <p><strong>Creată la:</strong>
        <?php echo date('d-m-Y', strtotime($row['created_at'])) . ' ' . date('H:i',
strtotime($row['created_at'])); ?>
```

Decizia de a reformata datele este în cazul acesta o bună practică și contribuie la o experiență a utilizatorului, formatul zi-luna-an fiind unui mai natural și intuitiv pentru utilizatorii din România.

3.1.3. Generarea unor chart-uri (canvas) pentru ilustrarea unor statistici, folosind un script

Acest script generează niște grafice (de tipul pie-chart) folosind biblioteca Chart.js. Datele necesare pentru elaborarea graficelor sunt extrase dintr-un fisier PHP care returnează un .json.

```
<script>
    fetch("get_stats.php")
      .then((response) => response.json())
      .then((data) => {
        function createChart(canvasId, chartLabel, labels, values) {
          new Chart(document.getElementById(canvasId), {
            type: "pie",
            data: {
              labels: labels,
              datasets: [
                {
                  label: chartLabel,
                  data: values,
                  backgroundColor: ["#ff6384", "#36a2eb", "#ffcd56",
"#4bc0c0"],
                },
              ],
            },
            options: {
              responsive: true,
              maintainAspectRatio: false
            }
          });
        }

        createChart("personalityChart", "Distribuție Personalitate",
data.personality.labels, data.personality.values);
        createChart("depressionChart", "Distribuție Depresie",
data.depression.labels, data.depression.values);
        createChart("serviceTypeChart", "Distribuție Servicii",
data.serviceType.labels, data.serviceType.values);
        createChart("sessionTypeChart", "Distribuție Ședințe",
data.sessionType.labels, data.sessionType.values);
      });

  </script>
```

Utilizăm `fetch("get_stats.php")` pentru a face o cerere HTTP către fișierul PHP, care returnează apoi datele în format JSON (practic o colecție de date). Apoi, cu `response.json()` convertim răspunsul într-un obiect JavaScript pentru a-l utiliza ulterior.

Se construiește funcția de creare a charturilor, care acceptă 4 parametrii (id-ul canvas-ului, eticheta graficului, valorile și etichetele pentru feliile graficului)

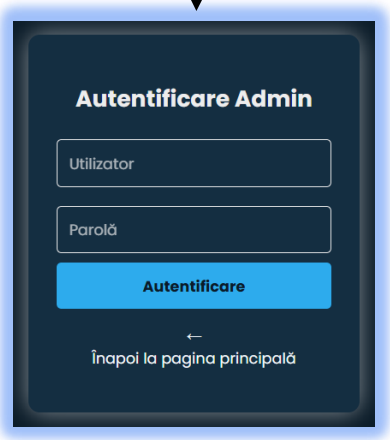
Ne asigurăm că graficele se vor adapta la dimensiunea containerului prin folosirea opțiunilor de adaptare responsive setate la „true” și menținerea aspectului setată „false.

3.1.4. Inițierea unei sesiuni în momentul în care ne logăm ca administrator

Acest mecanism contribuie la securitatea site-ului, protejând paginile sensibile și asigurând că doar persoanele autorizate pot accesa funcționalitățile de administrare. Avantajul acestei abordări, spre deosebire de cookies, este că sesiunile sunt gestionate de server și nu pot fi așa de ușor manipulate.

```
session_start();
if (!isset($_SESSION["admin"])) {
  header("Location: login.php");
  exit();
}
```

Prin `session_start()` se inițiază o sesiune existentă, ceea ce permite stocarea datelor despre utilizator pe server. Apoi, verifică dacă variabila de sesiune `$_SESSION["admin"]` este setată. Dacă utilizatorul nu este logat ca administrator, acesta este imediat redirecționat către pagina de login (login.php).



3.1.5. Imposibilitatea ștergerii contului de administrator cu care ești logat

Acest cod permite ștergerea unui cont de administrator din baza de date, asigurând în același timp că utilizatorul autenticat nu își poate șterge propriul cont. Pentru un plus de securitate, conexiunea folosește PDO, prevenind astfel atacuri de tip SQL Injection (interogări pregătite cu intenție malițioasă)

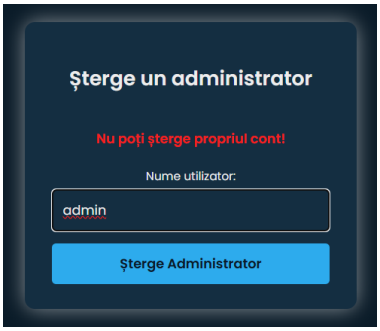
```
  $pdo = new PDO("mysql:host=localhost;dbname=licenta", "root",
"");
  $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

  if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $usernameToDelete = $_POST["username_to_delete"];

    if ($usernameToDelete == $_SESSION["admin"]) {
      $error = "Nu poți șterge propriul cont!";
    } else {

      $stmt = $pdo->prepare("DELETE FROM admins WHERE username =
?");
      $stmt->execute([$usernameToDelete]);
      $message = "Administrator șters cu succes!";
    }
```

Dacă cererea este de tip POST, verifică numele utilizatorului și, dacă este valid, execută ștergerea. În caz contrar (intenția de a șterge userul de administrator cu care s-a logat), afișează un mesaj de eroare.



3.1.6. Extragerea datelor privind programarile din baza de date pentru a genera un fisier PDF cu informațiile necesare.

Folosind biblioteca FPDF vom genera un fișier PDF în care să cuprindem sub formă tabelară ședințele de psihologie planificate pentru următoarele 2 săptămâni. Mai jos am ilustrat modul în care datele sunt extrase din baza de date.

```
while ($row = $result->fetch_assoc()) {
  $pdf->Cell(10, 10, $row['id'], 1);
  $pdf->Cell(40, 10, utf8_decode($row['name']), 1);
  $pdf->Cell(50, 10, utf8_decode($row['email']), 1);
  $pdf->Cell(30, 10, $row['telefon'], 1);
  $pdf->Cell(30, 10, date('d-m-Y', strtotime($row['date'])), 1);
  $pdf->Cell(30, 10, $row['time'], 1);
  $pdf->Ln();
}


$pdf->Output('D', "Raport_Programari_$today_formatted-
$next_14_days_formatted.pdf");
```

Printr-un loop while, preluăm fiecare rând din rezultatul interogării folosind fetch_assoc() și adaugă datele în tabelul PDF, aplicând formatare pentru caractere speciale și date. După inserarea datelor în tabel, fișierul este generat și descărcat automat sub un nume care include intervalul de timp al programărilor, facilitând accesarea și/sau distribuirea planificatorului într-un format mai profesional.

Raport Programari pentru perioada 29-03-2025 - 12-04-2025

ID	Nume	Email	Telefon	Data	Ora
62	Radu Mihai Pustan	andrada.terpe@yahoo.com	0742465847	29-03-2025	09:00:00
59	Radu Mihai Pustan	mp_radu@yahoo.com	0742465847	02-04-2025	09:00:00

Exemplu fisier PDF generat

3.1.7. Obținerea perioadelor disponibile pentru programare, cu eliminarea intervalelor blocate și/sau eliminarea perioadelor blocate de administrator pentru diverse motive (ex. concediu).

Prin acest fragment de cod extragem din baza de date atât programările existente, cât și perioadele blocate, urmând să le returneze în format JSON (colecție de date) pentru a fi utilizate în front-end. Aceste linii sunt esențiale pentru extragerea, manipularea și trimiterea datelor către client, fiind fundamentale pentru logica de disponibilitate a programărilor pe site.

```
$sqlAppointments = "SELECT `date`, `time` FROM appointments";
$resultAppointments = $conn->query($sqlAppointments);
$appointments = [];

if ($resultAppointments->num_rows > 0) {
  while ($row = $resultAppointments->fetch_assoc()) {
    $appointments[] = $row;
  }
}

$sqlBlocked = "SELECT `start_date`, `end_date` FROM blocked_days";
$resultBlocked = $conn->query($sqlBlocked);
$blocked_days = [];

if ($resultBlocked->num_rows > 0) {
  while ($row = $resultBlocked->fetch_assoc()) {
    $blocked_days[] = $row;
  }
}

$conn->close();

header('Content-Type: application/json');
echo json_encode([
  "appointments" => $appointments,
  "blocked_days" => $blocked_days
]);
```

Preluăm în prima fază toate datele și orele programărilor din tabela appointments și le stocăm într-un array. În mod similar, sunt obținute zilele blocate din tabela blocked_days.

După ce conexiunea cu baza de date este închisă, datele sunt trimise în format JSON către client.

Acest mecanism este esențial pentru afișarea în timp real a disponibilității programărilor, permițând interfației web să arate doar intervalele libere și să evite suprapunerea cu zilele deja ocupate sau indisponibile.

3.1.8. Extragerea datelor privind rezultatele testelor de depresie pentru a fi afișate ulterior în chart-uri dinamice.

Acest cod realizează o interogare complexă pentru a obține datele din tabela depresie_rezultate, care sunt esențiale pentru a analiza nivelul de depresie al utilizatorilor pe o perioadă de timp specificată (în acest caz, ultimele 30 de zile).

```
$sql2 = "SELECT depression_level, COUNT(*) AS total
FROM depresie_rezultate
         WHERE timestamp >= NOW() - INTERVAL 1 MONTH
         GROUP BY depression_level";
$result2 = $conn->query($sql2);
$depression = ["labels" => [], "values" => []];
while ($row2 = $result2->fetch_assoc()) {
  $depression["labels"][] = $row2["depression_level"];
  $depression["values"][] = $row2["total"];
}
```

După ce interogarea este executată, datele sunt procesate într-un array asociativ, unde:

- **labels** conține nivelurile de depresie (de exemplu, ușoară, moderată, severă),
- **values** conține numărul de persoane care se încadrează în fiecare nivel de depresie.

Aceste date sunt esențiale pentru a crea statistici vizuale sau rapoarte care reflectă distribuția nivelurilor de depresie într-o anumită perioadă, oferind o bază pentru analiza stării de sănătate mintală a utilizatorilor.

`fetch_assoc()` parcurge fiecare linie a rezultatelor obținute din interogare. Apoi, pentru fiecare linie, adaugă nivelul de depresie în array-ul labels și numărul total de persoane pentru acel nivel în array-ul values. Această abordare permite stocarea datelor într-o formă organizată, ușor de utilizat pentru crearea unor grafice sau rapoarte vizuale.

3.1.9. Logica de autentificare în platforma de administrator

Acest cod reprezintă logica de autentificare a unui utilizator pe baza unui nume de utilizator și a unei parole, utilizând sesiuni pentru a gestiona autentificarea.

```
session_start();
$pdo = new PDO("mysql:host=localhost;dbname=licenta",
"root", "");
if ($_SERVER["REQUEST_METHOD"] == "POST") {
  $username = $_POST["username"];
  $password = $_POST["password"];

  $stmt = $pdo->prepare("SELECT * FROM admins WHERE
username = ?");
  $stmt->execute([$username]);
  $user = $stmt->fetch();

  if ($user && password_verify($password,
$user["password"])) {
    $_SESSION["admin"] = $user["username"];
    header("Location: admin.php");
    exit();
  } else {
    $error = "Utilizator sau parolă incorectă!";
  }
```

După inițierea sesiunii, stabilim o conexiune cu baza de date folosind PDO (PHP Data Object), fiind mai sigur/flexibil comparative cu mysqli.

Verificăm dacă formularul de login a fost trimis prin metoda POST.

După preluarea userului/parolei din formular, căutam ca acestea să corespundă cu datele din tabela administratorilor folosind o interogare SQL.

Folosind `password_verify()` comparam parola introdusă cu parola criptată din baza de date, iar daca acestea coincid salvăm numele utilizatorului în sesiune și redirecționăm utilizatorului către pagina de administrare.

3.1.10. Popularea perioadelor/datelor/orelor disponibile pentru programări

Acest cod are rolul de a popula un selector de ore pentru programări, verificând disponibilitatea acestora în funcție de ziua aleasă și de programările deja existente. exemplu, dacă utilizatorul selectează o sâmbătă, programările vor fi disponibile între 9:00 și 14:00. După aceea, se face un apel către server pentru a obține lista de programări existente și zilele blocate, folosind fetch('get_appointments.php'). Dacă data aleasă este deja blocată (de exemplu, 5 martie între 10:00 și 12:00), opțiunile din selector vor fi limitate, iar utilizatorului i se va afișa mesajul "Această zi este blocată". De asemenea, se verifică pentru fiecare oră din intervalul disponibil dacă aceasta este deja rezervată; dacă nu este, ora este adăugată în lista de opțiuni. În cazul în care nu sunt ore disponibile, se va adăuga o opțiune cu mesajul "Nu mai sunt ore disponibile".

```
function populateTimeSlots(date) {
    const selectTime = document.getElementById("time");
    selectTime.innerHTML = '';

    const selectedDate = new Date(date);
    const dayOfWeek = selectedDate.getDay();

    let startHour, endHour;

    if (dayOfWeek === 0) {
        const option = document.createElement("option");
        option.textContent = "Nu se fac programări duminica";
        option.disabled = true;
        selectTime.appendChild(option);
        return;
    } else if (dayOfWeek === 6) {
        startHour = 9;
        endHour = 14;
    } else { // Luni - Vineri
        startHour = 9;
        endHour = 19;
    }

       fetch('get_appointments.php')
        .then(response => response.json())
        .then(data => {
            const { appointments, blocked_days } = data;

            const isDateBlocked = blocked_days.some(blocked => {
                const startDate = new Date(blocked.start_date);
                const endDate = new Date(blocked.end_date);
                return selectedDate >= startDate && selectedDate <= endDate;
            });

            if (isDateBlocked) {
                const option = document.createElement("option");
                option.textContent = "Nu există intervale disponibile în această zi.";
                option.disabled = true;
                selectTime.appendChild(option);
                return;
            }

            // verificăm disponibilitatea orelor
            for (let hour = startHour; hour < endHour; hour++) {
                const timeString = hour.toString().padStart(2, '0') + ":00";

                // verificăm dacă ora este deja ocupată
                const isBooked = appointments.some(appointment => {
                    const appointmentDate = new Date(appointment.date);
                    const appointmentTime = appointment.time;


                    const selectedDateString = selectedDate.toISOString().split('T')[0];
                    const appointmentTimeString = appointmentTime.substring(0, 5); // extragem
doar ora din formatul HH:MM

                    return appointmentDate.toISOString().split('T')[0] === selectedDateString &&
appointmentTimeString === timeString;
                });

                if (!isBooked) {
                    const option = document.createElement("option");
                    option.value = timeString;
                    option.textContent = timeString;
                    selectTime.appendChild(option);
                }
            }

            // dacă nu sunt ore disponibile, afișăm mesaj
            if (selectTime.options.length === 0) {
                const option = document.createElement("option");
                option.textContent = "Nu mai sunt ore disponibile";
                option.disabled = true;
                selectTime.appendChild(option);
            }
        })
        .catch(error => console.error("Eroare la preluarea programărilor:", error));
}
```

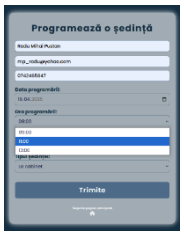
Pentru a popula doar intervalele orare dorite, vom stabili un „program de lucru" diferențiat pentru zilele săptămânii. Mai exact, duminică dezactivăm opțiunea de disponibilitate, sâmbăta stabilim un program redus iar în timpul săptămânii (L-V) un program mai stabil.

Extragem programările deja existente în baza de date folosind `fetch` apoi transformăm datele format json (colecție)

Verificăm dacă data este sau nu blocată, prin compararea cu datele din tabela blocked_days iar dacă data este blocată afișăm un mesaj de indisponibilitate.

Folosind un rationament similar, verificăm disponibilitatea orelor. Întâi verificăm dacă ora este deja ocupată prin compararea constantei isBooked cu datele din tabela appointments.

Comparăm progresiv data și apoi ora (prin extragerea orei din formatul HH:MM) iar dacă nu există orele nu sunt rezervate sunt afișate ca timeString. În caz contrar, afișăm un mesaj de de indisponibilitate.



3.1.11. Limitarea intervalului disponibil pentru programări la 21 zile în viitor

Acest fragment de cod are scopul de limitare a datei selectabile într-un câmp de tip date, astfel încât utilizatorul să poată selecta doar datele de la ziua curentă până la maxim 21 de zile în viitor. La încărcarea paginii (când evenimentul DOMContentLoaded este declanșat), scriptul obține data curentă (today) și creează o nouă dată pentru limita superioară (maxDate), care este setată la 21 de zile în viitor.

```
document.addEventListener("DOMContentLoaded", function () {
    const dateInput = document.getElementById("date");
    const today = new Date();
    const maxDate = new Date();
    maxDate.setDate(today.getDate() + 21);

    const todayFormatted =
today.toISOString().split('T')[0];
    const maxDateFormatted =
maxDate.toISOString().split('T')[0];

    dateInput.setAttribute("min", todayFormatted);
    dateInput.setAttribute("max", maxDateFormatted);
});
```

La încărcarea completă a paginii (când evenimentul DOMContentLoaded este declanșat), scriptul obține data curentă (today) și creează o nouă dată pentru limita superioară (maxDate), care este setată la 21 de zile în viitor.

Datele curente și limitele sunt formate în formatul corespunzător pentru un câmp date (YYYY-MM-DD) folosind metoda toISOString(), iar aceste valori sunt atribuite atributelor min și max ale câmpului date. Acest lucru garantează că utilizatorul nu poate selecta o dată anterioară zilei curente sau o dată mai îndepărtată de 21 de zile.

Captură de ecran cu formularul de programare online

3.1.12. Transmiterea datelor din formularul de programare către server prin Ajax

Acest fragment de cod JavaScript realizează transmiterea datelor unui formular către server folosind tehnologia AJAX (Asynchronous JavaScript and XML), care permite realizarea cererilor la server fără a reîncărca pagina.

```
    if (name && email && date && time && sessionType &&
onlyNumbers.test(telefon)) {
        const submitBtn = document.getElementById('submit');
        submitBtn.textContent = 'Se trimite...';
        fetch('salvare_programari.php', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/x-www-form-urlencoded',
            },
            body:
`name=${encodeURIComponent(name)}&email=${encodeURIComponent(email)}&telefo
n=${encodeURIComponent(telefon)}&date=${encodeURIComponent(date)}&time=${en
codeURIComponent(time)}&serviceType=${encodeURIComponent(serviceType)}&sess
ionType=${encodeURIComponent(sessionType)}`
        })
            .then(response => response.text())
            .then(data => {
                submitBtn.textContent = 'Trimite';
                submitBtn.disabled = true;
                const message = document.getElementById("message");
                message.innerHTML = data; // afișăm răspunsul de la server
                message.style.color = 'green';
                message.style.display = "block"; // facem vizibil mesajul


            })
            .catch(error => {
                console.error('Error:', error);
                const message = document.getElementById("message");
                message.textContent = "A apărut o eroare la salvarea
programării.";
                message.style.display = "block";
            });
    } else {
        document.getElementById("message").textContent = "Te rog
completează toate câmpurile!";
    }
});
```

Înainte de transmiterea efectivă, codul verifică dacă toate câmpurile din formular sunt completate corect, iar la expresia (onlyNumbers.test(telefon)) validăm dacă sunt introduse doar cifre.

Dacă se îndeplinesc condițiile acestea, textul butonului se schimba în „Se trimite” și se face o cerere de tip POST către salvare_programari.php.

După ce serverul procesează cererea, răspunsul este procesat în felul următor:
- Pentru succes primim un mesaj de succes
- Dacă apare o eroare, primim un mesaj de eroare

3.1.13. Generarea și procesarea testelor online (inclusiv salvarea rezultatelor în baza de date)

Acest sistem implică un chestionar interactiv pentru evaluarea depresiei, unde întrebările și răspunsurile sunt generate dinamic prin JavaScript.

```
<div class="container text-center">
    <h1>Test DEPRESIE</h1>
    <form id="quiz">
      <ul class="list-group">
        <!-- intrebarile sunt generate de javascript -->
      </ul>
    </form>
  </div>
```

Codul HTML conține un formular (<form id="quiz">) unde întrebările vor fi populate de JavaScript.

EXTRAS DIN depresie.js

```
function generateQuiz() {
      quizContainer.innerHTML = '';
      prompts.forEach((prompt, index) => {
          let questionHTML = `
              <li class="list-group-item">
                  <p class="prompt">${prompt.prompt}</p>
                  <div class="btn-group">`;
          prompt_values.forEach(value => {
              questionHTML += `
                  <button class="btn value-btn ${value.class}" data-weight="${value.weight}"
data-group="group${index}">${value.value}</button>`;
          });

          questionHTML += `
                  </div>
              </li>`;
          quizContainer.innerHTML += questionHTML;
      });
  }


  function handleButtonClick(event) {
      event.preventDefault();

      const target = event.target;
      if (target.classList.contains('value-btn')) {
          const group = target.dataset.group;


          document.querySelectorAll(`[data-group="${group}"]`).forEach(btn => {
              btn.classList.remove('active');
          });

          // adaugam clasa active la butonul apasat
          target.classList.add('active');
      }
  }
```

Funcția `generateQuiz()` creează dinamic întrebările pentru fiecare prompt din lista prompts. Fiecare întrebare este reprezentată de un element `<li>`. Pentru fiecare se creează un grup de butoane, care sunt asociate cu un atribut data-weight care indică punctajul asociat fiecăruia (ex. 0 pentru Total dezacord, 4 pentru Total acord).

În momentul apăsării unui buton funcția handleButtonClick() are rolul de a accepta doar un singur răspuns / întrebare (adăugând clasa active la butonul selectat). Acest lucru ne permite selectarea și înregistrarea pe baza punctajului asociat fiecărei opțiuni.

```
EXTRAS din depresie_data.js
var prompts = [
  {
      prompt: 'Mă simt trist(ă) sau abătut(ă).',
      class: 'group0',
      section: 'Evaluarea sentimentului de supărare/tristețe'
  },
  {
      prompt: 'Am observat că plâng mai ușor în ultima vreme.',
      class: 'group1',
      section: 'Evaluarea sentimentului de supărare/tristețe'
  },
  {
      prompt: 'Am observat că în ultimul timp iau decizii cu
dificultate.',
      class: 'group3',
      section: 'Evaluarea sentimentului de supărare/tristețe'
  },

  {
      etc.
  }
];

var prompt_values = [
  {
      value: 'Total dezacord',
      class: 'value-btn btn-strongly-disagree',
      weight: 0
  },
  {
      value: 'Dezacord',
      class: 'value-btn btn-disagree',
      weight: 1
  },
  {
      value: 'Neutru',
      class: 'value-btn',
      weight: 2
  },
  {
      value: 'De acord',
      class: 'value-btn btn-agree',
      weight: 3
  },
```

Am pus setul cu întrebările (prompts) într-un fisier separat depresie_data.js le-am grupat în așa fel încât codul să fie cât mai aerisit și ușor de citit, fără a încărca inutil fișierul care gestioneaza logica cu conținutul efectiv al întrebărilor/variantelor de răspuns.

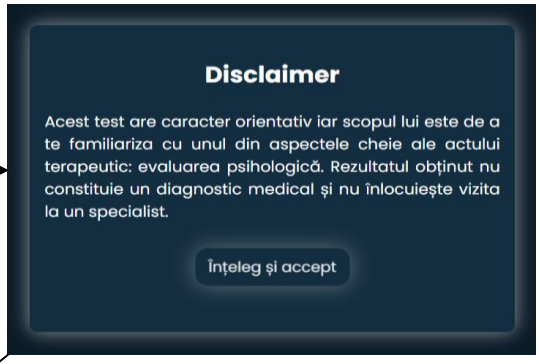
Variantele de răspuns (prompt_values) au fost asociate cu clase (pentru a putea fi stilizate) și cu câte un atribut pentru scorare (weight) astfel încât să oglindească o scală Likert cu 5 itemi. Prin acest demers am încercat să replic anumite standarde des folosite în domeniul statisticii și al analizei de date.

```
  function saveResultsToDatabase(total, resultMessage) {
    fetch("salvare_rezultate_personalitate.php", {
      method: "POST",
      headers: {
        "Content-Type": "application/x-www-form-urlencoded",
      },
      body:
`total=${total}&personality_type=${encodeURIComponent(resultMessage)}`
,
    })
      .then((response) => response.json())
      .then((data) => {
        if (data.success) {
          console.log("Rezultatul a fost salvat cu succes!");
        } else {
          console.error("Eroare la salvare:", data.message);
        }
      })
      .catch((error) => console.error("Eroare la request:", error));
  }
```

Salvarea datelor in baza de date, folosind metoda POST, cu specificarea conținutului prin Content-Type. Rezultatul este convertit ulterior în JSON.

Exemplificare pe front-end a rulării unui test psihologic în cadrul aplicației:

(1)
Accesarea testul din pagina index



(2)
Acceptarea consimțământului

(3)
Parcurgerea itemilor testului

(4)
Afișarea rezultatului

4. Evaluarea, testarea și validarea aplicației

4.1. Aspecte și precizări generale

În dezvoltarea oricărui produs software, etapa de testare joacă un rol foarte important, fiind cea care asigură că aplicația funcționează conform așteptărlor și poate fi utilizată în condiții reale fără a afecta experiența utilizatorului, securitatea datelor sau rapiditatea sistemului. Deși poate părea o etapă secundară în comparație cu designul sau programarea, testarea oferă certitudinea că produsul final respectă cerințele inițiale și se comportă stabil în fața unui număr mare de situații anticipate în faza de proiectare.

În cazul aplicației dezvoltate în cadrul acestei lucrări de licență, testarea a fost luată în calcul încă din faza de construcție a funcționalităților, fiind realizată gradual și constant, în paralel cu dezvoltarea propriu-zisă, după o abordare specifică metodologiei Agile. Procesul a presupus testarea manuală a aplicației în diferite condiții, simularea unor interacțiuni posibile din partea utilizatorilor, validarea datelor și controlul accesului, dar și evaluarea comportamentului aplicației în ceea ce privește interfața și compatibilitatea ei cu dispozitive mobile și rezistența în fața unor atacuri de natură mai comună.

În paginile următoare sunt prezentate detaliat tipurile de testări efectuate, scenariile simulate, erorile întâlnite și soluțiile implementate, precum și o sinteză a rezultatelor, care evidențiază măsura în care aplicația îndeplinește standardele de calitate necesare pentru a putea fi utilizată în mod real într-un cabinet psihologic.

4.1.1. Importanța testării în procesul de dezvoltare

De regulă, în procesul de creare a unui produs software, testarea are un rol foarte important și deseori neglijat. Prin intermediul acestui proces ne asigurăm că produsul final îndeplinește cerințele beneficiarului. Chiar dacă o aplicație web rulează fără erori vizibile nu înseamnă automat că aplicația este pregătită pentru a fi lansată online. Doar în urma unui proces mai amplu de testare am putea fi siguri că soft-ul se va comporta în modul în care intenționăm noi să o facă. Mai mult, testarea nu vizează doar detectarea și corectarea erorilor ci și o evaluarea parametrilor de performanță, siguranța datelor și eventual diferite caracteristici de compatibilitate între diverse medii de execuție sau chiar și adaptarea conținutului interfeței la diverse ecrane/rezoluții prin media query.

În cazul acestei lucrări, testarea a fost efectuată gradual, în parelel cu procesul de dezvoltare. Astfel, am reușit să observ imediat comportamentele disfuncționale/neconforme și am evitat acumularea de erori. Și în acest caz am lucrat conform metodologiei Agile. Cu alte cuvinte, testarea a fost realizată manual, direct în cadrul mediului de dezvoltare (XAMPP) și cu ajutorul instrumentelor puse la dispoziție de Chrome (Developer Tools). Altfel spus, am încercat să simulez diferite scenarii de utilizare (ex. dublarea programărilor sau completarea câmpurilor cu date nevalide) pentru a testa reacția sistemului.

4.1.2. Testarea funcțională

Pentru a mă asigura ca cerințele beneficiarului (creionate în faza de proiectare) sunt atinse, am testat rând pe rând fiecare funcționalitate a aplicației. Un exemplu mai concret ar fi la modulul de programare unde am testat ca validarea input-ului să nu permită alegerea datelor din trecut, interfața grafică să afișeze doar intervalele disponibile, alerga de confirmare a programării să fie afișată bine, respectiv erorile să fie afișate cu mesaje reprezentative.

Un exemplu concret de problemă identifică ar fi acela în care, la început, aplicația permitea salvarea unei programări chiar dacă formularul era incomplet (ex. nu selectam ora / lăsam câmpul pentru nume gol). Prin folosirea consolei din Developer Tools, am observat că formularul trimitea un request incomplet către server, iar partea de back-end nu valida în intregime input-ul. Pentru a remedia această situație am introdus o validare suplimentară în fișierul PHP care gestiona formularul astfel încât să nu permită trimiterea lui fără a fi completat integral.

4.1.3. Testarea logicii aplicației (back-end)

Fiind scris în PHP, back-end-ul aplicației a fost testat prin parcurgerea mai multor scenarii des întâlnite, de genul „Ce se intampla daca…?”. Câteva exemple ar fi:

- *Ar putea un utilizator să acceseze dashboard-ul de administrator fără să fie autentificat?*
- *Este posibil ca un utilizator să modifice URL-ul și să acceseze secțiuni care nu sunt destinate lui?*

Într-unul din cazuri, testând funcționalitatea acestui dashboard, am constatat că accesul în secțiunea destinată administratorului necesită autentificare la fiecare accesare. Să zicem că inchideai din greșeală pagina web, la repornire (după câteva secunde) se solicita din nou autentificarea, lucru care deveanea destul de supărător. Astfel, pentru a rezolva acest aspect am

implementat un sistem care păstra logarea în cadrul unei *sesiuni*, fără a necesita autentificări multiple succesive.

4.1.4. Testarea interfeței (front-end)

Pe partea de interfață am testat comportamentul fiecărui buton, am verificat partea de validare a formularelor în browser și afișarea corectă a mesajelor de stare. În Developer Tools am folosit consola pentru a verifica eventuale erori JavaScript, dar și rubrica din tab-ul Network pentru a mă asigura că cererile sunt trimise spre server și timpii de răspuns se clasează în parametrii normali.

Un exemplu de eroare identificată aici a fost legată de formularul de testare. Atunci când utilizatorul selecta mai multe răspunsuri sau lăsa anumite câmpuri necompletate, aplicația trimitea eronat datele către server. Din tab-ul Network am văzut că existau parametrii lipsă (ex: `q1[]=on&q1[]=on`). Pentru a rezolva această speță am restructurat formularul astfel încât răspunsurile să aiba un comportament nerelaționat, pentru a fi identificat clar și unic.

De asemenea, am petrecut foarte mult timp pentru a descoperi / cizela erorile ce țin de CSS pe partea de randare a componentelor, în special pe partea de dispozitive mobile, lucrând cu opțiunea de media query pentru a crea o experiență vizuală plăcută și foarte intuitive pentru toate tipurile de terminale. Aici am mizat pe principiul minimalismului și al eficienței, în sensul că fiecare funcționalitate trebuie să fie cât mai vizibilă și ușor de parcurs.

4.1.5. Testarea sistemelor de stocare/gestionare a bazei de date

Fiind un o aplicație software care manipulează date mai delicate, precum programări, rezultate la testări psihologice, am testat baza de date din perspectiva integrității și a securității pentru a evita anumite breșe/vulnerabilități.

În acese sens, am verificat ca toate câmpurile obligatorii să conțină atributul *required,* datele programărilor să fie vizibile și editabile doar de către administratorul aplicației (din dashboard), cererile SQL să fie protejate împotriva atacurilor de tip SQL Injection (prin mysqli_real_escape_string() și prepared statements (Buchmann R. 2020f).

Un alt test important pe care l-am realizat a fost încercarea de a trimite manual cereri de tip POST către server cu solicitări periculoase, de exemplu „DROP TABLE admins”. În urma verificărilor am constatat că sistemul refuză astfel de input-uri datorită validărilor implementate anterior pe sesiuni / autentificări / formulare.

4.1.6. Erori frecvent întâlnite în timpul dezvoltării

Pe măsură ce avansam cu dezvoltarea aplicației am întâlnit o serie de erori tipice proiectelor de acest gen. Câteva dintre cele mai des întâlnite au fost conflictele între validarea JavaScript și cea PHP care se manifestau prin faptul că uneori datele erau văzute ca valide de către browser dar erau refuzat de server (sau viceversa). Rezolvarea acestor situații a fost făcută prin omogenizarea / alinierea regulilor de validare pe ambele paliere.

Alteori, pe măsură ce baza de date era extinsă cu tot mai multe tabele, am constatat diverse lacune în ceea ce privește datele: fie existau lipsuri (ca urmare a validării neconforme a formularelor) fie diverse seturi de date incorecte salvate. Cele mai multe dintre ele scoteau în evidență neconcordanțe între numele câmpurilor din formular și coloanele tabelelor din baza de date.

Un alt aspect care a necesitat foarte multă atenție și timp a fost acela de afișare/randare pe dispozitivele mobile. Pornind de la mărimea fonturilor care trebuia ajustată permanent până la suprapunerea unor elemente care afectau semnificativ experiența de utilizare a platformei, toate au necesitat scrierea unor segmente extinse de cod în cadrul media-query-urilor. Totodată, pentru a îmbunătății percepția clienților, am încercat să limitez numărul de erori care afișau mesaje generice. Astfel, am încercat să introduc mesaje cât mai descriptive și care să facă trimitere exact la cauza erorii pentru a fi cât mai ușor de remediat de către utilizator.

În total, în timpul testării, au fost identificate și corectate peste 25 de erori, majoritatea în etapa de integrare a componentelor.

4.1.7. Metodologia de testare

În ceea ce privește ansamblul testelor efectuate, am ales să abordez acest proces cât mai organizat/sistematic și să parcurg niște criterii minimale care să ofere aplicației o verificare temeinică. În tabelul de mai jos am încercat să sumarizez/schematizez întreg demersul.

Criteriu testat	Grad de acoperire estimat	Modul testat	Metoda
Validarea formularelor	100%	Programare, Testare	Manual
Autentificare și sesiuni	100%	Toate modulele	Manual
Performanță încărcare pagini	85%	Frontend	DevTools / Network
Afișare corectă pe mobile	95%	Frontend	Accesare de pe device mobil
Manipulare date în DB	100%	Back-end + SQL	Testare directă
Securitate date (input filtering)	85%	Back-end	Manual / scenarii

Figura 10: Tabel privind estimarea gradului de acoperire a testelor efectuate

În ceea ce privește gradul de acoperire am făcut o estimare subiectivă, încercând să transmit faptul că majoritatea funcționalităților aplicației au fost testate cu atenție luând în calcul modul în care un utilizator normal ar interacționa cu sistemul web.

4.2. Sinteză

În ansamblul ei, testarea aplicației mi-a permis să identific din timp erorile și să le remediez rând pe rând, odată cu dezvoltarea platformei web. Respectând și în acest caz metodologia Agile și uneltele de dezvoltare/testare (precum Developer tools), am încercat să abordez întreg procesul de testare într-un mod cât se poate de organizat / sistematic. Pornind de la testarea funcțională/logică și până la testarea adaptării conținutului pe diverse interfețe (mobile/desktop) am reușit să obțin un site web stabil și foarte intuitive.

Chiar și așa, sunt conștient de faptul că odată ce va fi folosită în condiții reale (nesimulate), va fi necesară o nouă analiză și cel mai probabil diverse intervenții ulterioare. Pentru că oricât m-aș strădui eu să găsesc bug-uri și alte disfuncționalități, îmi este imposibil să anticipez multitudinea de modalități în care utilizatorii vor exploata platforma. Astfel, după finalizarea proiectului, aplicația va rămâne deschisă pentru varii actualizări și/sau îmbunătățiri. În acest sens iau în calcul crearea unui modul de facturare și a unui modul de plăți online pentru a ușura munca birocratică a psihologului.

Concluzionând acest capitol pot spune că activitățile de testare reprezintă o component esențială a oricărui demers de dezvoltare pe zona de software. În cazul meu, această activititate a contribuit în foarte mare măsură la îmbunătățirea experienței pe care o are atât clientul cât și administratorul.

Concluzii și direcții de viitor

Reflecții personale asupra proiectului

Pentru mine această lucrare nu a fost doar o provocare din punct de vedere tehnic (eu neavând nicio experiență anterioară în zona de programare) ci și o șansă de a veni în spijinul unui antreprenor care ar putea beneficia într-un mod cât se poate de real de acest produs software. Având și o pregătire anterioară în domeniul psihologiei, am încercat să creez o simbioză benefică între cele 2 domenii astfel încât acestea să se eleveze reciproc. Astfel, am încercat să folosesc diferite principii psihologice (ex. tendința de a folosi calea care întâmpină cea mai mică rezistență) în crearea unei arhitecturi software intuitive, respectiv să folosesc tool-urile digitale pentru a transforma serviciile psihologice în niște activități foarte accesibile (un exemplu fiind modului de programare online, care facilitează întreg procesul și reduce stresul mental al primei interacțiuni sociale).

Practic, aflându-se la intersecția dintre IT, psihologie, economie comportamentală și servicii sociale, acest demers mi-a demonstrat că digitalizarea nu este un proces rezervat doar pentru companiile cu mulți bani, ci poate fi accesibilă și companiilor mai mici (sau chiar PFA-urilor), cum este cazul unui cabinet de psihologie individual.

În fapt, la baza acestei lucrări de licență a stat o idee simplă, conform căreia timpul este o resursă foarte prețioasă, mai ales pentru cei care prestează servicii de natură intangibilă (mai ales cei care își gestionează singuri activitatea profesională). Exact din acest motiv am ținut în mod special să dezvolt o soluție software care nu doar să funcționeze ci și să aducă un beneficiu real și vizibil în viața beneficiarului – fie că ne raportăm la client sau la administratorul aplicației web.

Sunt conștient că acesta nu este un produs finit apt pentru a fi *comercializat*, dar în același timp reprezintă de departe cel mai ambițios proiect pe care l-am întreprins în cei 3 ani de studii. Pentru mine reprezintă un mare succes faptul că am reușit să construiesc o aplicație funcțională (în tot ansamblul ei), care reușește să digitalizeze/automatizeze o parte semnificativă din sarcinile administrative ale cabinetului psihologic gestionat de beneficiar. Cele mai reprezentative funcționalități, care scutesc psihologul de mult timp/efort, sunt cele de programare online, testare psihologică și oferirea unui dashboard de control pentru administrator. În fapt, având în vedere că acest tip de abordare digitalizată este foarte rar întâlnită (excepție făcând clinicile mari) în zona municipiului Cluj-Napoca, consider că am adus o contribuție reală prin acest proiect care aduce un plus de valoare net.

Direcții de viitor

Toate programele / soft-urile sunt, prin natura lor, maleabile și deschise. După etapele de proiectare, testare și eventual lansare, o aplicație devine un organism complex, care necesită supraveghere și îmbunătățire constantă. Pentru a face față rigorilor și așteptărilor tot mai mari ale clienților, orice

aplicație trebuie să fie capabilă să se extindă și să se adapteze evoluției mediului înconjurător: atât cel digital unde tehnologiile evolueaza constant cât și cel economic unde influențele și interferențele sunt bidirecționale. Așadar, raportându-mă la website-ul construit de mine, am conturat câteva direcții clare de evoluție, printre care se regăsesc următoarele:

a) Îmbunătățirea procesului de autentificare prin introducerea unor mecanisme mai avansate precum autentificarea în 2 pași (2FA) sau eventual logarea prin emiterea unui cod temporar trimis pe email. Aceste măsuri ar crește nivelul de securitate însă ar necesita și un angajament de timp suplimentar din partea celor care îl parcurg. Abordarea acestui obiectiv de viitor ar avea sens în cazul în care traficul ar depăși nivelul anticipat, justificând astfel aceste măsuri suplimentare de siguranță.

b) Integrarea unor servicii externe de tipul SMS, prin care platforma să trimită notificări/remindere clienților / psihologului pentru a reduce rata de neprezentare și eventualele erori. Acest serviciu ar emula comportamentul marilor platforme (ex. Regina Maria) și ar crea o percepție de profesionalism asupra cabinetului.

c) Extinderea modului de testare prin integrarea unui număr mai crescut de teste aplicabile prin intermediul platformei. Deoarece în prezent aplicația oferă doar 2 tipuri de teste (un screening pentru depresie și un test de personalitate) consider că pe viitor ar fi necesară o diversificare a acestora în așa fel încât să abordeze mai multe paliere ale evaluării psihologice. Cu toate astea, aici există anumite limitări care ar putea îngreuna procesul de extindere a acestui modul. Cea mai mare se referă la faptul că psihologii pot folosi aceste instrumente doar fiind în posesia unei licențe oficială care atestă dreptul de folosire. Chiar și așa, aceștia nu au dreptul să facă publice aceste instrumente lucru care complică destul de mult integrarea lor într-o platformă online.

d) Crearea unui sistem de plăți online prin integrarea unui servicii de procesare a tranzacțiilor precum PayPal sau Netopia. Un astfel de modul ar eficientiza foarte mult procesul/interacțiunea economică dintre client și psiholog, reducând cuantumul încasărilor în cash. Chiar dacă PFA-urile au un regim fiscal în care beneficiarul are acces permanent la banii proveniți din cabinet (fără a fi necesară distribuirea de dividende), o astfel de funcționalitate ar oferi un plus de valoare și o percepție mai favorabilă din partea clienților.

e) Îmbunătățirea modulului de statistici / metrice din dashboard-ul administratorului astfel încât să cuprindă o arie mai mare de informații utile. Acestea ar putea juca rolul unui tablou de comandă asemănător cu cele dezvoltate în Tableau Desktop în marile corporații. Pornind de la contorizarea vizualizărilor pe site și până la diverse grafice / distribuții care să ilustreze tendințele de navigare pe website, toate ar contribui la luarea unor decizii administrative mai bine documentate.

f) Continuarea optimizării pentru utilizatorii de dispozitive mobile prin crearea unei aplicații dedicate disponibile în MagazinPlay. Chiar dacă în acest moment aplicația este responsivă la mărimea displayului pe care este afișată (prin media query), o eventuală aplicație dedicată ar crește și mai mult accesibilitatea serviciilor. Această soluție reprezintă un țel îndepărtat care ar avea sens doar în cazul în care acest business devine unul de foarte mare succes. În acest fel, aplicația ar putea deveni scalabilă și să deservească mai mulți psihologi, fiecare având un cont propriu cu un portal administrativ distinct.

Ce am învățat pe parcursul dezvoltării acestui proiect

Chiar dacă proiectul final este unul cât se poate de funcțional, procesul de dezvoltare a fost unul foarte provocator și plin de lecții. În primul rând, fiind doar un student fără vreo experiență anterioară în zona programării web, m-am văzut nevoit să parcurg în timp scurt un traseu de învățare abrupt. Avantajul a fost că majoritatea tehnologiilor/limbajelor folosite au fost parcurse în cadrul facultății în primii 2 ani de studiu (HTML, CSS, JavaScript, SQL, PHP). Acest lucru mi-a permis să încep procesul de dezvoltare relativ rapid (majoritatea arhitecturii de bază fiind scrise în vacanța de vară dintre anul 2 și 3). Ulterior, pe parcursul ultimului an de studii m-am axat pe armonizarea, îmbunătățirea și extinderea funcționalităților astfel încât proiectul să fie unul cât se poate de coerent.

Probabil cea mai importantă lecție în acest demers a fost aceea din care am înțeles că dezvoltarea oricărui produs software nu necesită doar timp și diferite cunoștințe tehnice ci o înțelegere a ansamblului unui sistem și o planificare atentă a etapelor de proiectare și implementare. Fiind nevoit astfel să mă adaptez continuu, mi-am dat seama cât de importante sunt etapele de testare și debugging. Acestea, deși presupun mult timp, pot face diferența între o platformă web rudimentară și una fiabilă/intuitivă.

Un alt aspect pe care l-am urmărit cu interes, mai ales în perioada pre-dezvoltare, a fost cel legat de înțelegerea și evoluția contextului economic și al pieței serviciilor de sănătate mentală din zona în care activează cabinetul. Mai mult, făcând această aplicație pentru soția mea, am învățat nu doar să dezvolt o platformă digitală, ci să construiesc un produs software care să îndeplinească cerințele unui beneficiar cu păreri, idei și exigențe proprii. Totodată, având o oarecare experiență anterioară în gestionarea unui cabinet și a unei clientele anterioare, am ținut neapărat să fac din această aplicație una foarte intuitivă, aspectuoasă și care să inspire încredere și profesionalism.

Nu în ultimul rând, din analiza concurenței din piața locală am ajuns la concluzia că în era digitalizării diferența dintre progres și stagnare o face capacitatea de a te adapta și a integra soluții și tehnologii moderne care să crească expunerea și zona de acoperire a serviciilor prestate. Aceste argumente devin foarte importante mai ales pentru profesioniștii independenți (ex. PFA-uri), unde orice mic detaliu poate face o diferență mare în piața în care operează.

Elemente de valoare adăugată a lucrării

De departe cea mai reprezentativă contribuție a acestei lucrări o reprezintă integrarea unor funcționalități digitale avansate (modulul de programare online, modulul de testare psihologică și dashboard-ul pentru administratorul) la cerințele și nevoile unui cabinet de psihologie cu resurse tehnice și economice limitate.

Prin acest proiect am domonstrat că aceste funcționalități nu sunt rezervate doar pentru clinicile mari, ci pot fi adaptate specificului unui cabinet mic, printr-un cumul de cunoștințe tehnice (dobândite în urma unui efort nu foarte crescut) și multă perseverență. Practic, în cazul acesta, valoarea adăugată nu stă în tehnologia în sine, ci în felul în care aceasta este folosită pentru a servi unei activități din ce în ce mai necesară: psihoterapia. În acest fel, prin digitalizare, psihologul economisește mult timp iar clientul beneficiază de o experiență de navigare intuitivă și o interacțiune mult mai accesibilă cu funcționalitățile esențiale care preced orice act terapeutic: programarea și evaluarea inițială.

În cele din urmă, eu sunt de părere că această lucrare de licență demonstrează fezabililitatea unei abordări colaborative, în care un viitor informatician poate contribui într-o mare măsură la elevarea activității unui profesionist care activează într-un domeniu cu specific umanist (psiholog în acest caz). Prin această colaborare/sinergie am reușit nu doar să dezvolt o aplicație funcțională și intuitivă, ci și să învăț cum tehnologia, mai ales cea din domeniul IT, poate avea un impact concret și pozitiv în viața reală.

Bibliografie

- Ariely, D. (2010). *Irațional în mod previzibil: Forțele ascunse care ne influențează deciziile* (trad. M. Stan). Editura Publica. pp. 63–95.
- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., & Thomas, D. (2001). *Manifesto for Agile Software Development*. Agile Alliance. pp. 1–3. Preluat de pe: https://agilemanifesto.org/
- Buchmann, R. (2024). Suport de curs: Testarea produselor software, Cluj-Napoca
- Freeman, E., & Robson, E. (2014). Head First HTML and CSS (2nd edition); O'Reilly Media
- Global Market Insights. (2023). *Digital psychotherapeutics market size & share report, 2032*. Preluat de pe: https://www.gminsights.com/industry-analysis/digital-psychotherapeutics-market/market-size
- Hailey, D., Roine, R., & Ohinmaa, A. (2019). *The effectiveness of telemental health applications: A review.* The Canadian Journal of Psychiatry, 64(5), pp. 321–329.
- Harari, Y. N. (2017). *Homo deus: O scurtă istorie a viitorului* (trad. D. Butucea,). Iași: Polirom. pp. 102–135.
- Kesrouani, C. (2023, octombrie 14). *Online psychotherapy: 5 benefits*. Carla Kesrouani. Preluat de pe: https://carlakesrouani.com/2023/10/online-psychotherapy-5-benefits/
- Kahneman, D. (2012). *Gândire rapidă, gândire lentă* (trad. V. Răzvan). București: Ed. Publica. pp. 45–87.
- McFarland, D.S. (2015). Web Development with HTML, CSS, Javascript and jQuery Set.
- Moudatsou, S. (2024). *Evaluating the effectiveness of teletherapy for mental health during the COVID-19 pandemic*. *Journal of Behavioral Health*, 27(5), pp. 711–715.
- Turk, D., France, R., & Rumpe, B. (2014). Assumptions underlying agile software development processes. *arXiv preprint*. arXiv:1409.6610. pp. 1–10.

Anexe

Anexa 1: Chestionar de screening al depresiei disponibil în aplicație

Test DEPRESIE

Mă simt trist(ă) sau abătut(ă).
Total dezacord | Dezacord | Neutru | De acord | Total de acord

Am observat că plâng mai ușor în ultima vreme.
Total dezacord | Dezacord | Neutru | De acord | Total de acord

Am observat că în ultimul timp iau decizii cu dificultate.
Total dezacord | Dezacord | Neutru | De acord | Total de acord

Resimt deseori sentimentul de vinovăție/rușine.
Total dezacord | Dezacord | Neutru | De acord | Total de acord

Evit să îmi petrec timpul cu familia, prietenii sau colegii.
Total dezacord | Dezacord | Neutru | De acord | Total de acord

Am dificultăți în a mă motiva să încep o activitate/proiect personal.
Total dezacord | Dezacord | Neutru | De acord | Total de acord

Nu mai găsesc bucurie sau satisfacție în viața de zi cu zi.
Total dezacord | Dezacord | Neutru | De acord | Total de acord

Am probleme cu somnul, fie că nu pot dormi, fie că dorm prea mult.
Total dezacord | Dezacord | Neutru | De acord | Total de acord

Mă confrunt cu o schimbare a apetitului: fie am mai puțină poftă de mâncare, fie mai multă.
Total dezacord | Dezacord | Neutru | De acord | Total de acord

Libido-ul meu este scăzut.
Total dezacord | Dezacord | Neutru | De acord | Total de acord

Mă gândesc la moduri de a-mi face rău.
Total dezacord | Dezacord | Neutru | De acord | Total de acord

Simt ca viața mea este o povară.
Total dezacord | Dezacord | Neutru | De acord | Total de acord

Doresc să îmi pun capăt vieții.
Total dezacord | Dezacord | Neutru | De acord | Total de acord

Vezi rezultat

Anexa 2: Chestionar de personalitate al depresiei disponibil în aplicație

Test PERSONALITATE

Îmi este dificil să mă prezint oamenilor.

Total de acord | De acord | Neutru | Dezacord | Total dezacord

Mă pierd atât de mult în gândurile mele, încât ignor sau uit ce se întâmplă în jurul meu.

Total de acord | De acord | Neutru | Dezacord | Total dezacord

De obicei, nu inițiez conversații.

Total de acord | De acord | Neutru | Dezacord | Total dezacord

Prefer să nu interacționez cu oameni care par furioși sau supărați.

Total de acord | De acord | Neutru | Dezacord | Total dezacord

Îmi aleg prietenii cu atenție.

Total de acord | De acord | Neutru | Dezacord | Total dezacord

Îmi este dificil să spun povești despre mine.

Total de acord | De acord | Neutru | Dezacord | Total dezacord

De obicei, sunt foarte motivat(ă) și plin(ă) de energie.

Total de acord | De acord | Neutru | Dezacord | Total dezacord

Îmi este ușor să mă alătur unui grup de oameni și să intru în conversație.

Total de acord | De acord | Neutru | Dezacord | Total dezacord

A fi adaptabil este mai important decât a fi organizat.

Total de acord | De acord | Neutru | Dezacord | Total dezacord

Îmi pasă mai mult să mă asigur că nimeni nu se supără decât să câștig o dezbatere.

Total de acord | De acord | Neutru | Dezacord | Total dezacord

Adesea nu simt că trebuie să mă justific în fața oamenilor.

Total de acord | De acord | Neutru | Dezacord | Total dezacord

Aș prefera să improvizez decât să petrec timp elaborând un plan detaliat.

Total de acord | De acord | Neutru | Dezacord | Total dezacord

Vezi rezultatul